**UNIVERSITÀ POLITECNICA DELLE MARCHE**

**DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE**

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

**Insegnamento di New Generation Databases**




# Anomaly detection su dati biometrici

## Relazione di progetto

DOCENTE

**Prof.ssa Claudia Diamantini**

CANDIDATI

**Andrea Altieri**

**Andrea Flaiani**

Anno accademico 2026/2027

# Indice


**1 Introduzione e dominio applicativo** **1**

1.1 Contesto clinico e problematiche dei dati intraoperatori . . . . . 1
1.2 Scelta del dataset VitalDB e coorte sperimentale. . . . . 2
1.3 Impostazione architetturale e struttura dell'elaborato . . . . . 3

**2 Architettura della pipeline dati** **5**

2.1 Il modello medallion a tre livelli . . . . . 5
2.2 Livello bronze: acquisizione e storicizzazione dei dati grezzi. . . . . 6
2.3 Livello silver: bonifica fisiologica, imputazione e partizionamento . . . . . 7
2.4 Evoluzione del modello dati da tabella Parquet a documento BSON . . . . . 11

**3 Archiviazione e governance con MongoDB Time Series** **13**

3.1 Archiviazione temporale nel livello gold . . . . . 13
3.2 Analisi sperimentale di spazio su disco, latenza e valori mancanti . . . . . 14
3.3 Architettura di data governance e gestione transazionale . . . . . 16
3.3.1 Catalogo dei casi, contratti di schema e metadati delle tracce . . . . . 16
3.3.2 Protocollo di caricamento a due fasi e recupero automatico . . . . . 19
3.4 Pipeline di aggregazione e ottimizzazione delle interrogazioni . . . . . 20

**4 Modelli di anomaly detection** **22**

4.1 Impostazione dell'approccio ibrido. . . . . 22
4.2 Estrazione dal livello gold e formulazione delle regole cliniche. . . . . 22
4.3 Ingegnerizzazione delle feature temporali e imputazione locale . . . . . 23
4.4 Modelli non supervisionati: isolation forest e autoencoder LSTM . . . . . 24
4.4.1 Partizionamento casuale nello spazio a otto dimensioni con isolation forest. . . . . 24
4.4.2 Architettura ricorrente dell'autoencoder LSTM e gestione della memoria . . . . . 25
4.5 Valutazione sperimentale sui 500 casi e persistenza degli allarmi . . . . . 27

**5 Ingegnerizzazione del sistema, interfaccia grafica e conclusioni** **31**

5.1 Servizi REST con FastAPI . . . . . 31
5.2 Dashboard web per l'esplorazione clinica e architetturale . . . . . 32
5.2.1 Vista di telemetria clinica e analisi del consenso . . . . . 33
5.2.2 Vista di monitoraggio dell'architettura e dei benchmark. . . . . 33
5.3 Containerizzazione Docker, orchestrazione e test automatici . . . . . 34
5.4 Conclusioni, analisi dei limiti e sviluppi futuri . . . . . 35

Capitolo 1

# Introduzione e dominio applicativo

## 1.1 Contesto clinico e problematiche dei dati intraoperatori

Durante un intervento chirurgico i monitor multiparametrici di sala operatoria acquisiscono in modo continuo i parametri vitali del paziente, tra cui la frequenza cardiaca, la saturazione periferica dell'ossigeno e la pressione arteriosa. Queste serie temporali rappresentano uno strumento essenziale per intercettare precocemente episodi di instabilità emodinamica, ma la loro gestione informatica e l'analisi automatica su larga scala pongono diverse difficoltà strutturali. In primo luogo, le registrazioni reali sono costantemente esposte a **rumore e artefatti strumentali**: il movimento del paziente sul letto operatorio, le interferenze elettromagnetiche generate dal bisturi elettrico o il distacco temporaneo di un sensore possono produrre dei picchi fisiologicamente impossibili, come frequenze cardiache superiori a $250$ bpm o letture pressorie nulle, oltre a frequenti interruzioni del segnale.

A questa rumorosità si affianca l'**asimmetria fisiologica e strutturale delle frequenze di campionamento** tra i diversi canali di monitoraggio. I sensori ottici (come il fotopletismografo a luce rossa e infrarossa per la saturazione d'ossigeno $SpO_2$) e i conduttori elettrocardiografici per la frequenza cardiaca (HR) sono dispositivi non invasivi passivi che intercettano l'attività cardiopolmonare battito per battito, consentendo al monitor di estrarre un campione numerico continuo ogni secondo ($1$ Hz) senza interruzioni e senza arrecare alcun disagio al paziente.

Al contrario, la pressione arteriosa non invasiva (**NIBP**) viene rilevata mediante un manicotto pneumatico oscillometrico posizionato attorno al braccio del paziente. Il monitor avvia il gonfiaggio della camera d'aria fino a superare la pressione sistolica per occludere temporaneamente l'arteria brachiale, per poi eseguire un graduale sgonfiaggio misurando le micro-oscillazioni pressorie trasmesse dalla parete vascolare. Questo ciclo fisico richiede dai $30$ ai $40$ secondi di elaborazione e **non può in alcun modo essere continuativo**: mantenere il bracciale costantemente compresso attorno all'arto bloccherebbe la perfusione sanguigna distale, provocando ischemia acuta dei tessuti, stasi venosa e gravi lesioni neuropatiche da compressione a carico del nervo mediano e ulnare. Per tale ragione, durante un intervento chirurgico il bracciale viene attivato in modo intermittente (tipicamente ogni $2$, $3$ o $5$ minuti), rimanendo completamente sgonfio e inerte per la quasi totalità del tempo.

Quando questi flussi eterogenei devono essere allineati su un'unica linea temporale con passo discreto di un secondo per correlare i parametri vitali e alimentare gli algoritmi di calcolo, la disparità di campionamento si traduce in una marcata **sparsità strutturale** della matrice dati: mentre le due colonne di battito e ossigeno hanno un valore quasi ad ogni riga, le tre colonne della pressione (`NIBP_SBP`, `NIBP_DBP` e `NIBP_MBP`) restano vuote e segnate come `NaN` (*Not a Number*, dato mancante) per centinaia di secondi tra un gonfiaggio e l'altro. Poiché tre parametri su cinque dipendono proprio dal bracciale pressorio, nei dati grezzi oltre il $66\%$ delle celle totali risulta inizialmente vuoto: non per un guasto del monitor, ma per il normale funzionamento fisico degli strumenti. Questo aspetto ci ha spinto a studiare strategie specifiche per riempire i buchi in modo sensato durante la pulizia dei dati nel livello silver.

Infine, è importante sottolineare che nei contesti ospedalieri reali si riscontra la quasi totale **assenza di etichette cliniche puntuali**: raramente un medico annota manualmente ogni singolo secondo di un intervento distinguendo le fasi di normalità fisiologica dagli eventi avversi, rendendo di fatto impraticabile l'addestramento diretto di modelli diagnostici puramente supervisionati.

L'obiettivo di questo lavoro è la progettazione e lo sviluppo di una piattaforma completa in grado di governare l'intero ciclo di vita dei tracciati biometrici intraoperatori: dall'acquisizione dei file grezzi alla loro pulizia, dall'archiviazione ottimizzata su un database temporale non relazionale fino al rilevamento automatico delle anomalie e alla consultazione interattiva tramite interfaccia web.

## 1.2 Scelta del dataset VitalDB e coorte sperimentale

Per costruire e validare la piattaforma su tracciati clinici reali è stato adottato **VitalDB**, una banca dati pubblica raccolta presso il dipartimento di anestesiologia del Seoul National University Hospital che raccoglie le registrazioni multiparametriche ad alta risoluzione di oltre $6.300$ interventi chirurgici. In particolare, VitalDB mette a disposizione un'interfaccia HTTP pubblica e la libreria ufficiale `vitaldb` per Python, consentendo di automatizzare via codice lo scaricamento dei casi clinici senza complicazioni.

All'interno di VitalDB sono presenti sia le forme d'onda continue ad alta frequenza ($100$–$500$ Hz), ovvero il disegno continuo delle onde elettriche che scorrono sullo schermo del monitor, sia le serie numeriche già estratte dal monitor multiparametrico alla frequenza di $1$ Hz. Nel progetto si è scelto di operare esclusivamente sulle serie numeriche a $1$ Hz registrate dal monitor `Solar8000` poiché l'obiettivo centrale del lavoro risiede nell'architettura della base di dati, nella modellazione documentale e nella qualità stessa del dato: l'elaborazione delle forme d'onda grezze avrebbe spostato il baricentro verso algoritmi di filtraggio in frequenza ed estrazione dei picchi, senza apportare benefici significativi alla gestione dei dati temporali.

Nello specifico, l'analisi prende in esame cinque parametri vitali complementari:

- **Frequenza cardiaca (`Solar8000/HR`):** espressa in battiti al minuto (bpm) e acquisita in modo continuo ogni secondo ($1$ Hz);
- **Saturazione periferica dell'ossigeno (`Solar8000/PLETH_SPO2`):** espressa in percentuale (%) e acquisita anch'essa in continuo a $1$ Hz;
- **Pressione arteriosa sistolica non invasiva (`Solar8000/NIBP_SBP`):** il valore massimo di pressione arteriosa, misurato in millimetri di mercurio (mmHg) dal bracciale ad intervalli intermittenti di $2$–$5$ minuti;
- **Pressione arteriosa diastolica non invasiva (`Solar8000/NIBP_DBP`):** il valore minimo di pressione arteriosa tra due battiti, misurato in mmHg insieme alla sistolica;
- **Pressione arteriosa media non invasiva (`Solar8000/NIBP_MBP`):** espressa in mmHg e approssimabile tramite la relazione $MBP \approx DBP + \frac{1}{3}(SBP - DBP)$. Dal punto di vista clinico rappresenta l'indicatore principale con cui l'anestesista valuta l'afflusso di sangue agli organi vitali, considerando la soglia di $MBP < 65$ mmHg come limite critico al di sotto del quale cresce il rischio di sofferenza per reni, cuore e cervello.

Accanto alle cinque tracce biometriche, il sistema acquisisce i metadati anagrafici e ospedalieri dei pazienti, salvati nel file `clinical_data.parquet` con informazioni quali l'età, il sesso e il reparto chirurgico `department`, insieme ai referti delle analisi ematochimiche perioperatorie archiviati in file Parquet `lab_data.parquet`, un formato che permette di conservare i dati in modo compatto e di

mantenere correttamente i tipi dei valori, per esempio numeri e date. La sperimentazione su cui è stata validata e misurata l'intera architettura comprende **500 interventi chirurgici completi**, che corrispondono a **5.565.384 righe temporali grezze** estratte dalla sorgente e a **5.402.386 righe** corrispondenti agli istanti in cui era presente almeno un parametro vitale. Nel complesso, i dati rappresentano oltre 1.500 ore di monitoraggio in sala operatoria.

La dinamica dell'asimmetria di campionamento e l'origine fisiologica della sparsità sono schematizzate visivamente nella Figura 1.1, che mette a confronto sincronizzato le due corsie temporali su una finestra di $5$ minuti ($300$ s) di rilevazione.

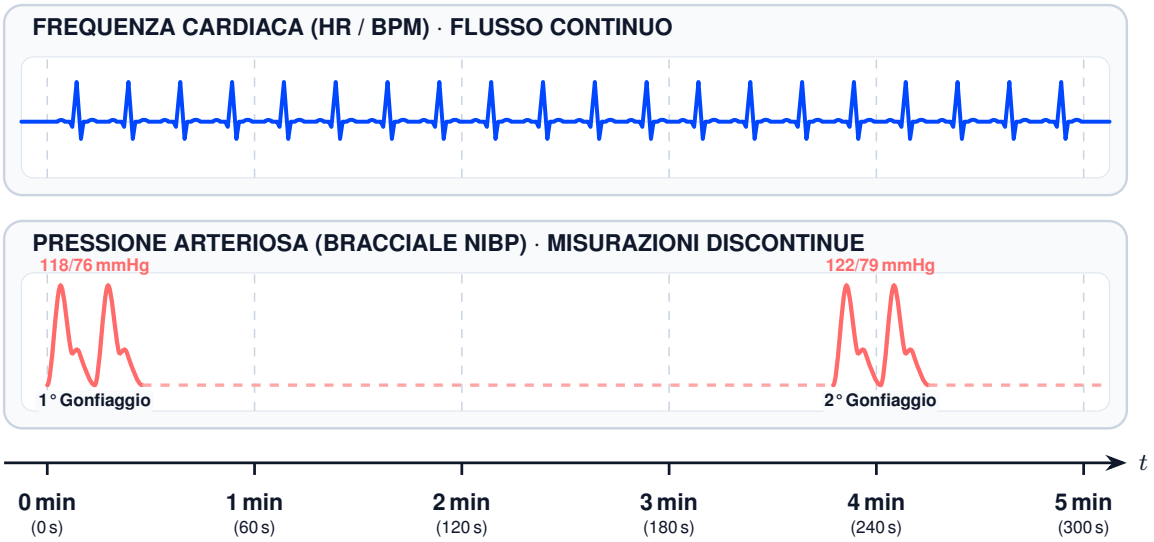



**Figura 1.1:** Confronto sincronizzato su finestra temporale di $5$ minuti ($300$ s) tra il monitoraggio continuo a $1$ Hz (HR ed ECG/$SpO_2$) e la rilevazione intermittente oscillometrica del bracciale (NIBP): evidenza del vuoto di rilevazione ($> 200$ secondi di celle NaN non misurate tra due cicli di gonfiaggio del bracciale), della sparsità strutturale nei dati grezzi e della propagazione controllata con tecnica LOCF (Last Observation Carried Forward) nel livello silver.

## 1.3 Impostazione architetturale e struttura dell'elaborato

Il sistema è stato progettato seguendo il paradigma dell'**architettura medallion**, che organizza il percorso dei dati in tre livelli progressivi denominati **bronze**, **silver** e **gold**. Questa separazione netta tra l'acquisizione immutabile dei tracciati grezzi, la fase intermedia di pulizia e arricchimento su disco locale e lo strato finale di servizio sul database ha garantito la piena riproducibilità degli esperimenti: ogni modifica alle regole di filtraggio degli outlier o di imputazione temporale ha potuto essere applicata rigenerando i livelli silver e gold a partire dai file locali in pochi minuti, senza mai ripetere il lento scaricamento di rete dai server di VitalDB.

Come accennato, per la memorizzazione su disco nei primi due livelli è stato adottato il formato colonnare binario **Apache Parquet** al posto dei tradizionali file testuali CSV, ottenendo una forte riduzione dello spazio occupato, il mantenimento nativo dei tipi numerici in virgola mobile a 64 bit e la possibilità di partizionare fisicamente i file del livello silver in sottocartelle per reparto chirurgico (`department=<REPARTO>`). Lo strato gold è stato invece affidato a una collezione temporale nativa di **MongoDB 8.0** (`vital_signals`), che raggruppa i secondi contigui dello stesso paziente in blocchi compressi chiamati **bucket**. MongoDB gestisce internamente l'organizzazione delle misure e permette di affiancare ai segnali informazioni di contesto, come i metadati clinici.

Poiché l'architettura creata unisce file Parquet su disco e un database NoSQL, abbiamo voluto affiancarle un sistema di controllo e tracciabilità (**data governance**) ispirato ai moderni **data**

**lakehouse**. Nelle grandi aziende questo compito è svolto da tecnologie come Delta Lake o Apache Iceberg appoggiate a cluster Apache Spark, che però sarebbero state eccessive per un progetto su macchina singola e non avrebbero potuto gestire dall'interno le collezioni Time Series di MongoDB. Le garanzie di integrità, tracciabilità delle esecuzioni tramite `run_id` e recupero automatico da arresti anomali sono state quindi implementate combinando una collezione di catalogo (`registry`) validata da uno schema formale con un protocollo di caricamento a due fasi in Python.

Sopra il livello gold opera infine un motore di **anomaly detection** ibrido che unisce due regole mediche deterministiche, basate sullo **shock index** e sull'ipotensione severa corretta per l'età della misura pressoria, con due algoritmi di **machine learning** non supervisionato, ovvero un modello ad alberi **isolation forest** e una rete neurale ricorrente **autoencoder LSTM** realizzata tramite la libreria PyTorch. L'intera piattaforma è infine esposta tramite un servizio REST sviluppato con **FastAPI** e una dashboard web interattiva in HTML5, CSS3 e JavaScript, il tutto racchiuso in due container **Docker** orchestrati automaticamente.

L'esposizione dei capitoli successivi ricalca il flusso logico appena tracciato: il **Capitolo 2** dettaglia l'implementazione dei livelli bronze e silver e la trasformazione dei dati da tabella Parquet a documento BSON; il **Capitolo 3** approfondisce l'archiviazione sul livello gold in MongoDB 8.0, i risultati sperimentali sui tempi di risposta e sullo spazio su disco e i meccanismi di governance; il **Capitolo 4** descrive la formulazione delle regole cliniche, l'ingegnerizzazione delle feature, i modelli non supervisionati e la loro valutazione; il **Capitolo 5** illustra infine i servizi API, l'interfaccia grafica, la suite di test automatici e un bilancio critico dei limiti architetturali con i possibili sviluppi futuri.

Capitolo 2

# Architettura della pipeline dati

## 2.1 Il modello medallion a tre livelli

Per elaborare oltre cinque milioni di secondi di monitoraggio biometrico mantenendo una distinzione rigorosa tra il dato originale e le trasformazioni successive, la pipeline è stata strutturata, come anticipato nel Capitolo 1, secondo l'**architettura medallion**, un modello di progettazione molto usato nei data lake che organizza i dati in tre livelli successivi: **bronze**, **silver** e **gold**.

Invece di scaricare i tracciati dall'interfaccia remota di VitalDB, applicare a mano le correzioni in memoria centrale e inserire direttamente i record nel database, ogni livello persiste il proprio stato con un ruolo ben delimitato:

- **Livello bronze (dati grezzi):** ha l'unico compito di scaricare i tracciati da VitalDB e salvarli su disco locale esattamente come arrivano dalla sorgente, senza toccare o filtrare alcun valore;
- **Livello silver (pulizia e arricchimento):** legge i file locali del livello bronze, individua i valori fuori scala dovuti a errori dei sensori, riempie i buchi temporali con regole diverse a seconda del tipo di segnale, aggiunge le colonne di controllo della qualità e organizza i file in sottocartelle divise per reparto chirurgico;
- **Livello gold (database temporale):** trasforma le tabelle pulite del livello silver in documenti BSON e li carica nella collezione Time Series di MongoDB 8.0, rendendoli pronti per le interrogazioni rapide della dashboard web e per i modelli di anomaly detection.

L'architettura complessiva della pipeline e il relativo flusso sequenziale di elaborazione, dall'ingestione remota alla visualizzazione nella dashboard web, sono illustrati nella Figura 2.1.

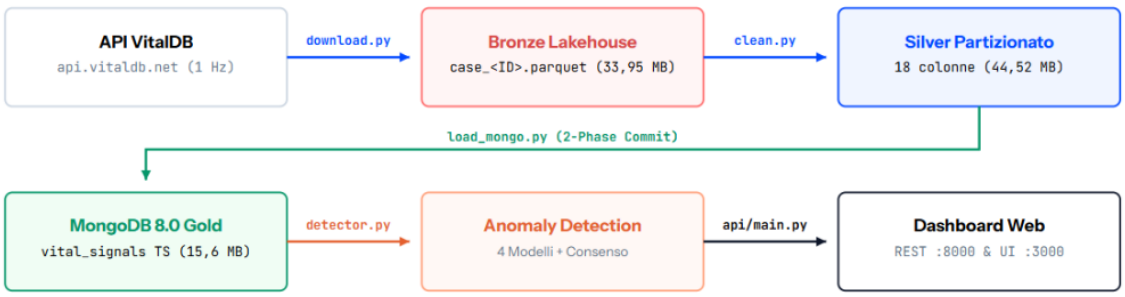



**Figura 2.1:** Architettura end-to-end della pipeline dati e flusso di elaborazione: ingestione bronze, bonifica silver, caricamento gold su MongoDB 8.0, rilevamento delle anomalie e interfaccia web.

Il motivo di questa architettura a più livelli sta nei vantaggi pratici che offre. In primo luogo, se durante lo sviluppo cambiavamo una regola di pulizia, potevamo rigenerare il livello silver leggendo i file bronze già presenti sul disco, senza dover rifare le $500$ richieste di rete a VitalDB, una per ogni paziente. In secondo luogo, operazioni come l'interpolazione dei buchi temporali o

la propagazione in avanti della pressione richiedono di scorrere intere colonne di numeri consecutivi: farlo in Python con Pandas e NumPy sui file Parquet del livello silver è immediato, mentre eseguirlo direttamente dentro MongoDB tramite query di aggregazione su milioni di documenti sarebbe stato molto più complesso.

## 2.2 Livello bronze: acquisizione e storicizzazione dei dati grezzi

Il primo stadio della pipeline è realizzato nello script `src/bronze/download.py`, il cui scopo è portare in locale, per ciascun paziente, i valori dei cinque parametri vitali. Il primo passo è capire quali interventi sono utilizzabili: la funzione `find_available_cases` interroga VitalDB tramite il metodo `vitaldb.find_cases(tracks)` e restituisce solo gli interventi in cui sono presenti tutte e cinque le tracce richieste (`HR`, `PLETH_SPO2`, `NIBP_SBP`, `NIBP_DBP` e `NIBP_MBP`), perché un paziente a cui mancasse anche un solo sensore non permetterebbe di applicare le stesse regole e gli stessi modelli sulle stesse colonne. Da questo elenco selezioniamo al massimo $500$ interventi (`MAX_CASES = 500`, definito in `src/config.py`): sono i primi dell'elenco restituito da VitalDB e non un campione casuale, quindi non vanno considerati rappresentativi della popolazione, ma bastano per avere milioni di campioni gestibili su un singolo computer, in un progetto centrato sull'architettura dei dati e non su uno studio clinico. Per ciascun intervento la funzione `download_case` chiede a VitalDB i valori delle cinque tracce campionati a intervalli regolari di un secondo (`SAMPLING_INTERVAL = 1.0`), cioè un valore per ogni sensore ad ogni secondo. Il risultato arriva come una semplice griglia di numeri, con una riga per ogni secondo e una colonna per ogni sensore, ma senza indicare a quale secondo corrisponda ciascuna riga. Lo script la trasforma allora in una tabella Pandas e aggiunge una sesta colonna `Time` che numera i secondi trascorsi dall'inizio dell'intervento ($0, 1, 2, \dots$): in questo modo ogni riga ha un istante esplicito e possiamo sempre verificare che le righe siano distanziate di un secondo, proprietà che ci servirà più avanti nel livello silver. Infine la tabella viene salvata nel file `data/bronze/case_<ID>.parquet`. Per evitare che un'interruzione improvvisa lasci sul disco file incompleti o illeggibili, il salvataggio avviene sempre su un file temporaneo con suffisso `.tmp`, che viene rinominato in modo atomico nella destinazione definitiva solo al termine della scrittura.

La scelta di adottare **Apache Parquet** fin dal livello bronze nasce da motivi pratici di spazio e velocità: se avessimo salvato $5,56$ milioni di righe in $500$ file CSV tradizionali, tutti i numeri sarebbero stati scritti come semplice testo, occupando molto più spazio su disco e costringendo Python, ad ogni lettura, a reinterpretare milioni di stringhe per riconvertirle in numeri decimali (`float64`). Parquet salva invece i dati già in formato binario, organizzati per colonna e compressi automaticamente (tramite l'algoritmo Snappy): in questo modo l'intero archivio bronze dei $500$ pazienti occupa appena **33,95 MB** e viene ricaricato in memoria da `pandas.read_parquet` in pochi millisecondi, mantenendo intatti i tipi di tutte le colonne.

Oltre ai tracciati secondo per secondo dei singoli interventi, lo script del livello bronze scarica anche due tabelle generali che descrivono i pazienti: `clinical_data.parquet` (con età, sesso e reparto chirurgico) e `lab_data.parquet` (con i risultati degli esami del sangue). Tra tutte le informazioni disponibili ne usiamo in particolare tre, ovvero **età, sesso e reparto** (`department`): servono a dare un contesto a ciascun tracciato, cioè a sapere che tipo di paziente stiamo guardando, e a poter poi organizzare e filtrare i casi per reparto, cosa che faremo nel livello silver e nel database del livello gold. Poiché queste due tabelle non hanno la stessa importanza per il resto della pipeline, abbiamo gestito in modo diverso il caso in cui il loro scaricamento dovesse fallire:

- **Dati clinici e di reparto (`clinical_data.parquet`):** questo file è indispensabile perché, nel passaggio successivo, il livello silver usa proprio la colonna `department` per smistare i file dei pazienti nelle rispettive sottocartelle di reparto. Per questo, se il download non riesce nemmeno dopo aver provato due canali diversi, lo script si ferma subito (politica **fail-fast**,

`sys.exit(1)`): andare avanti senza reparti significherebbe mettere tutti i pazienti in un'unica cartella generica `department=UNKNOWN`, perdendo il senso del partizionamento.

- **Esami di laboratorio (`lab_data.parquet`):** al contrario, i referti degli esami del sangue vengono salvati nel livello bronze solo per avere lo storico completo del dataset, ma non vengono usati nei calcoli delle anomalie a $1$ Hz. Per questo motivo, nella funzione `download_lab_data` se il server remoto non risponde lo script non blocca l'intera pipeline (politica **graceful fallback**), ma annota semplicemente che il file non è disponibile (`unavailable`) e prosegue il lavoro.

Poiché la pipeline può essere rilanciata più volte, abbiamo reso ogni esecuzione tracciabile e ripetibile senza sprechi. Ogni esecuzione di `download.py` genera un identificativo univoco (`run_id`) tramite la funzione `config.get_or_create_run_id(force_new=True)`, memorizzato nel file `data/.current_run_id` e riportato nel registro `data/download_manifest.json` insieme alla versione della pipeline (`pipeline_version: "1.1.0"`) e all'intervallo di campionamento. Per ciascuno dei $500$ interventi, il manifest indica se il file è stato scaricato (`downloaded`), se era già presente ed è stato riutilizzato (`skipped_existing`) oppure se si è verificato un errore (`failed`). Quando lo script viene rilanciato su una cartella già popolata, prima di contrassegnare un file esistente come `skipped_existing` apre il relativo Parquet e verifica che questo contenga tutte le colonne attese e che i primi due campioni (`Time.iloc[1] - Time.iloc[0]`) siano distanziati di un secondo, come previsto; se i controlli non sono superati, il caso viene scaricato di nuovo. Il manifest viene poi letto dal livello silver per sapere quali casi elaborare, e lo stesso `run_id` ricompare nel catalogo del database descritto nel Capitolo 3, in modo da collegare ogni dato alla propria esecuzione.

## 2.3 Livello silver: bonifica fisiologica, imputazione e partizionamento

In questa fase partiamo dai dati grezzi salvati nel livello bronze, in cui il $66,3\%$ delle celle numeriche è vuoto (`NaN`) e in cui compaiono anche valori incompatibili con la fisiologia umana, generati da errori o disturbi dei sensori. Il modulo `src/silver/clean.py` trasforma ciascun caso applicando una sequenza ordinata di operazioni che pulisce le serie storiche senza cancellare le tracce dei valori sbagliati.

Un aspetto metodologico cruciale riguarda il mantenimento della griglia temporale regolare. Come visto nel livello bronze, ogni file contiene una riga per ogni singolo secondo dell'intervento (colonna `Time` $= 0, 1, 2, \ldots$), cioè un campione al secondo ($1$ Hz): se in un certo secondo il monitor non ha registrato nulla (per esempio perché i cavi sono scollegati), la riga esiste comunque ma contiene solo valori `NaN`. Questa regolarità è importante perché le funzioni di Pandas che usiamo per riempire i buchi ragionano contando le **righe**, non i secondi: quando scriviamo `max_gap=5` o `ffill(limit=300)` intendiamo "$5$ secondi" e "$300$ secondi" solo perché ogni riga vale esattamente un secondo. Nella funzione `clean_case` la griglia completa viene quindi preservata fino al termine di tutti i calcoli, scartando le righe in cui tutti e cinque i parametri vitali sono simultaneamente nulli (`dropna(subset=vital_cols, how='all')`) soltanto come ultima istruzione: se le togliessimo all'inizio, un'interruzione di dieci minuti dei sensori sparirebbe dalla tabella e i valori prima e dopo il distacco finirebbero su righe adiacenti, come se fossero passati pochi secondi. Sulla coorte di $500$ interventi, questa scelta comporta la rimozione di **162.998 righe inattive su 5.565.384** (**2,93%**), corrispondenti quasi interamente ai minuti iniziali e finali di sala operatoria in cui il monitor è acceso ma i cavi non sono ancora collegati al paziente.

Prima di riempire i buchi, lo script cerca nei dati i valori impossibili dal punto di vista fisiologico (gli **artefatti**), come ad esempio una frequenza cardiaca di $320$ bpm causata dal bisturi elettrico. Per riconoscerli confronta ogni misura con un intervallo di valori plausibili per quel parametro,

riportato nella Tabella 2.1. Gli intervalli sono volutamente larghi: servono a scartare solo i valori fisicamente impossibili, non quelli gravi ma realistici (come una saturazione del $70\%$ o una pressione media di $50$ mmHg), che sono proprio gli eventi che le regole e i modelli devono poi individuare. Questi limiti sono scritti una volta sola, nel dizionario `TRACK_SPECS` del modulo `src/config.py`, e tutti i moduli del progetto leggono da lì: lo script di pulizia del livello silver, per riconoscere gli outlier, e lo script del livello gold, che li salva nella collezione `track_metadata` del database. Nelle prime versioni del progetto gli stessi limiti erano invece scritti a mano in più file e capitava di modificarli in un punto dimenticandosi dell'altro, ottenendo soglie diverse tra pulizia e database; avere un'unica sorgente evita alla radice questo problema.

| Traccia VitalDB | Flag di qualità nel livello silver | Intervallo valido (TRACK_SPECS) | Unità |
|---|---|---|---|
| `Solar8000/HR` | `HR_outlier`, `HR_imputed` | $20 - 250$ | bpm |
| `Solar8000/PLETH_SPO2` | `SPO2_outlier`, `SPO2_imputed` | $50 - 100$ | % |
| `Solar8000/NIBP_SBP` | `SBP_outlier`, `SBP_imputed` | $40 - 250$ | mmHg |
| `Solar8000/NIBP_DBP` | `DBP_outlier`, `DBP_imputed` | $10 - 180$ | mmHg |
| `Solar8000/NIBP_MBP` | `MBP_outlier`, `MBP_imputed` | $20 - 220$ | mmHg |

**Tabella 2.1:** Intervalli di validità fisiologica definiti in `TRACK_SPECS` e relativi flag associati nel livello silver.

Quando un valore osservato cade al di fuori dell'intervallo indicato nella Tabella 2.1 (per esempio una frequenza cardiaca di $320$ bpm causata dal bisturi elettrico), non lo cancelliamo dalla tabella ma lo teniamo nella cella, impostando a `True` il corrispondente flag booleano `*_outlier`. Lo facciamo per la **tracciabilità e il controllo della qualità del dato**: se lo sostituissimo subito con `NaN`, non potremmo più distinguere un sensore scollegato (nessuna misura) da un sensore che ha misurato un valore impossibile (misura disturbata), non potremmo contare quanti artefatti ci sono per paziente nel report di qualità e non potremmo verificare a posteriori se i limiti scelti sono adeguati. Il valore sbagliato non inquina comunque i calcoli successivi: i moduli di analisi lo trasformano in `NaN` nel momento in cui leggono i dati dal livello gold.

Il controllo viene eseguito sui dati grezzi *prima* di riempire i buchi, e durante il riempimento le celle outlier vengono nascoste solo temporaneamente, come se fossero `NaN` (`raw_col.mask(out_mask)`); subito dopo il valore originale torna nella sua cella. Questo ordine evita due problemi. Immaginiamo che il bracciale misuri una sistolica di $120$ mmHg e che, al gonfiaggio successivo, per un errore di lettura registri $10$ mmHg: se riempissimo i buchi senza nascondere questo valore, il $10$ verrebbe copiato in avanti per tutti i secondi fino alla misura seguente, trasformando una sola lettura sbagliata in centinaia di righe sbagliate (e centinaia di outlier contati nel report). Nascondendolo prima, invece, al suo posto viene riportata l'ultima misura valida ($120$ mmHg), mentre la lettura anomala resta registrata una sola volta, nella sua riga, con il flag `SBP_outlier` a `True`.

Una volta isolati gli artefatti, l'imputazione dei valori mancanti viene differenziata in funzione della fisica dei sensori. Per i segnali continui ad alta frequenza, ovvero `HR` e `PLETH_SPO2`, viene utilizzata la funzione `interpolate_short_gaps(series, max_gap=5)`. A differenza del metodo standard `.interpolate(limit=5)` di Pandas, che riempie comunque i primi cinque secondi anche all'inizio di una lunga disconnessione del sensore generando una coda artificiale, la funzione implementata misura preliminarmente l'ampiezza di ciascuna sequenza di `NaN` e ricostruisce per interpolazione lineare esclusivamente le interruzioni interne di durata inferiore o uguale a $5$ secondi comprese tra due letture valide, registrando l'operazione nei flag `HR_imputed` e `SPO2_imputed`. Il limite di $5$ secondi è una scelta prudente: in un intervallo così breve frequenza cardiaca e saturazione non possono cambiare in modo significativo, quindi una retta tra le due misure vicine è una buona

approssimazione, mentre su buchi più lunghi staremmo inventando dati che il sensore non ha mai misurato.

Per le tre componenti della pressione arteriosa non invasiva (`NIBP_SBP`, `NIBP_DBP` e `NIBP_MBP`), misurate dal bracciale solo ogni $2$–$5$ minuti, interpolare non avrebbe senso: tra due gonfiaggi non sappiamo cosa faccia la pressione e una retta inventerebbe un andamento mai misurato. In sala operatoria, invece, il medico considera valida l'ultima pressione rilevata finché non ne arriva una nuova, e noi riproduciamo questo comportamento con la tecnica **LOCF** (*last observation carried forward*, "porta avanti l'ultima osservazione") tramite `ffill(limit=300)`, che copia in avanti l'ultima misura valida non-outlier per al massimo $300$ secondi e attiva i flag `SBP_imputed`, `DBP_imputed` e `MBP_imputed`. Il limite di cinque minuti è un compromesso: corrisponde all'intervallo più lungo tra due misure del bracciale, quindi copre l'intero ciclo normale ed evita di lasciare buchi nelle tre colonne (è uno dei motivi per cui i valori mancanti scendono dal $66,3\%$ al $26,3\%$, come vedremo nel Capitolo 3), ma oltre quella soglia una pressione non è più rappresentativa dello stato attuale del paziente e il valore torna a `NaN`. Va precisato che i $300$ secondi servono a fornire ai modelli una matrice il più possibile completa: le regole cliniche del Capitolo 4, più severe, usano soltanto pressioni più recenti di $120$ secondi.

Per sapere in ogni istante quanto è "vecchia" la pressione che stiamo leggendo, calcoliamo in ogni riga la colonna `nibp_age_s`, che misura i secondi trascorsi dall'ultima misura reale del bracciale (`nibp_age_s` $= t - t_{\text{last_cuff}}$); una lettura outlier non conta come misura valida, quindi in quel caso il contatore continua a crescere. Nei primissimi secondi dell'intervento, prima del primo gonfiaggio, non esiste ancora nessuna misura da cui contare, e assegniamo quindi il valore convenzionale `9999.0`. Usiamo un numero enorme, e non $0$ o `NaN`, per due motivi: $0$ significherebbe "misurata proprio adesso", cioè il contrario della realtà, mentre `NaN` lascerebbe un buco nella colonna; $9999.0$ supera invece qualsiasi soglia di freschezza (come i $120$ secondi delle regole cliniche) e quei secondi vengono scartati automaticamente, senza bisogno di casi particolari. L'effetto complessivo della pulizia, osservato sul segnale cardiaco, è visibile nella Figura 2.2, mentre il codice che la realizza è riportato nel Listato 2.1.

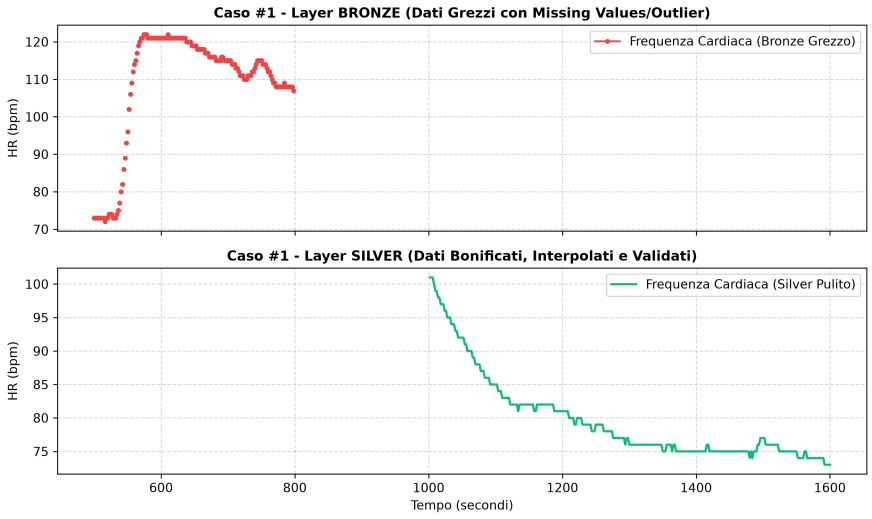



**Figura 2.2:** Confronto tra il tracciato grezzo della frequenza cardiaca nel livello bronze e il segnale bonificato nel livello silver.

```
// Codice estratto da src/silver/clean.py
# 1. Flag outlier calcolati sui dati grezzi (da config.TRACK_SPECS) prima di imputare
for track_name, flag_col in outlier_map.items():
    spec = track_specs.get(track_name, {})
    val = df_clean[track_name]
    df_clean[flag_col] = val.notna() & ((val < spec["min"]) | (val > spec["max"]))

# 2. Segnali continui (HR e SpO2): interpola solo buchi interni <= 5s senza propagare outlier
clean_hr, hr_imp = interpolate_short_gaps(raw_hr.mask(out_hr), max_gap=5)
df_clean['Solar8000/HR'] = clean_hr.where(~out_hr, raw_hr)
df_clean['HR_imputed'] = hr_imp & (~out_hr)

# 3. Segnali NIBP: calcolo nibp_age_s (sentinella 9999.0) e LOCF max 300s per traccia
last_measure_time = time_series.where(any_valid_nibp).ffill()
df_clean['nibp_age_s'] = (time_series - last_measure_time).fillna(9999.0).astype(float)
for t, o, imp_col in present_nibp:
    raw_col = df_clean[t]
    out_mask = df_clean[o]
    was_na = raw_col.isna()
    filled_col = raw_col.mask(out_mask).ffill(limit=300)
    df_clean[t] = filled_col.where(~out_mask, raw_col)
    df_clean[imp_col] = was_na & df_clean[t].notna()

# 4. Scarta solo alla fine le righe con tutte e 5 le tracce vitali contemporaneamente nulle
df_clean = df_clean.dropna(subset=vital_cols, how='all').copy()
```

**Listato 2.1:** Rilevamento preventivo degli outlier, interpolazione dei buchi brevi e propagazione LOCF in `src/silver/clean.py`.

Terminata la pulizia, ogni tabella viene salvata come file Parquet, ma non tutti nella stessa cartella: i file vengono divisi in sottocartelle, una per ogni reparto chirurgico, secondo la convenzione di nome `department=<REPARTO>` (detta **partizionamento Hive**):

`data/silver/department=<REPARTO>/case_<ID>.parquet`

Il reparto di un paziente non è scritto nel tracciato del monitor, quindi lo ricaviamo da `clinical_data.parquet`, scaricato nel livello bronze: lo script costruisce un dizionario che associa a ogni `case_id` il suo `department`. Gli identificativi vengono convertiti da `numpy.int64` (il tipo con cui li legge Pandas) al tipo `int` standard di Python, altrimenti la ricerca nel dizionario non troverebbe il reparto anche se il numero è lo stesso. Se per un paziente il reparto manca, il file finisce nella cartella `department=UNKNOWN`. Dividere i file per reparto serve a leggere solo i dati che interessano: per analizzare una singola specialità (come chirurgia generale o chirurgia toracica) basta aprire la sua sottocartella invece di tutti i $500$ file.

Nella stessa fase lo script tiene in ordine anche i risultati. All'avvio, la funzione `process_all_cases` rimuove dalla cartella `data/silver/` gli eventuali file di esecuzioni precedenti con un numero diverso di pazienti, così che i conteggi riflettano sempre e solo l'esecuzione corrente. Al termine produce un report di qualità, `data/quality_report.json`, con per ogni caso le righe scartate, gli outlier trovati e i valori ancora mancanti; prima di sovrascriverlo, archivia la copia dell'esecuzione precedente nella cartella `data/quality_history/`, così da poter confrontare nel tempo gli effetti di

eventuali modifiche alle regole di pulizia. Questo report è anche la base delle metriche di qualità del Capitolo 3.

## 2.4 Evoluzione del modello dati da tabella Parquet a documento BSON

L'attraversamento dei tre livelli dell'architettura medallion comporta una progressiva evoluzione della struttura e della rappresentazione fisica dei dati. Nel livello bronze ogni file Parquet presenta una struttura tabellare minima composta da **6 colonne**, corrispondenti all'istante temporale `Time` e alle cinque tracce biometriche grezze. Passando al livello silver, la tabella si espande a **18 colonne**: alle sei variabili originarie si affiancano l'identificativo `case_id`, i cinque flag booleani di anomalia strumentale (`HR_outlier`, `SPO2_outlier`, `SBP_outlier`, `DBP_outlier`, `MBP_outlier`), i cinque flag booleani di ricostruzione temporale (`HR_imputed`, `SPO2_imputed`, `SBP_imputed`, `DBP_imputed`, `MBP_imputed`) e il contatore continuo di invecchiamento pressorio `nibp_age_s`. Proprio l'introduzione di queste dodici colonne aggiuntive di tracciabilità, unita alla sostituzione di milioni di celle `NaN` con valori numerici propagati tramite LOCF, spiega per quale motivo i file Parquet del livello silver occupino su disco più spazio rispetto al livello bronze (**44,52 MB** contro **33,95 MB**, con un incremento del **31,1%**).

Nel passaggio finale al livello gold, realizzato dalla funzione `build_records` in `src/gold/load_mongo.py`, la rappresentazione tabellare lascia il posto al modello documentale gerarchico **BSON** proprio di MongoDB 8.0, con un'occupazione fisica di **15,60 MB** (più $2,56$ MB di indici; il motivo di questa riduzione è spiegato nel Capitolo 3). Come illustrato nel Listato 2.2, ciascun secondo attivo viene strutturato in quattro sezioni distinte: il campo radice `timestamp`, ottenuto convertendo i secondi relativi `Time` in un oggetto `ISODate` riferito alla data convenzionale `2020-01-01T00:00:00Z` (VitalDB fornisce solo i secondi trascorsi dall'inizio dell'intervento, senza la data reale, mentre le collezioni Time Series richiedono un campo temporale di tipo data); il sotto-documento `metadata`, che raccoglie gli attributi invarianti del paziente (`case_id`, `sensor_name`, `department`, `age` e `sex`) e funge da chiave di raggruppamento (`metaField`) per i bucket della collezione temporale; il dizionario `metrics`, che memorizza le misure fisiologiche sostituendo il carattere `/` con `_` nei nomi delle chiavi e omettendo fisicamente qualsiasi parametro risultato nullo in quel secondo; e infine il sotto-documento `quality_flags`, che preserva tutti i dieci flag booleani e l'età `nibp_age_s` a beneficio dei moduli di rilevamento delle anomalie a valle. Come questi documenti vengono caricati nel database con garanzie di integrità è l'argomento del Capitolo 3.

```
// Documento BSON archiviato nella collezione 'vital_signals'
{
  "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
  "timestamp": ISODate("2020-01-01T08:15:30Z"),
  "metadata": {
    "case_id": 1,
    "sensor_name": "Solar8000",
    "department": "General surgery",
    "age": 64,
    "sex": "M"
  },
  "metrics": {
    "Solar8000_HR": 72.0,
    "Solar8000_PLETH_SPO2": 98.0,
    "Solar8000_NIBP_SBP": 120.0,
    "Solar8000_NIBP_DBP": 80.0,
    "Solar8000_NIBP_MBP": 93.3
  },
  "quality_flags": {
    "HR_outlier": false, "SPO2_outlier": false,
    "SBP_outlier": false, "DBP_outlier": false, "MBP_outlier": false,
    "HR_imputed": false, "SPO2_imputed": false,
    "SBP_imputed": true, "DBP_imputed": true, "MBP_imputed": true,
    "nibp_age_s": 14.0
  }
}
```

**Listato 2.2:** Struttura del documento BSON inserito nella collezione temporale `vital_signals` del livello gold.

Capitolo 3

# Archiviazione e governance con MongoDB Time Series

## 3.1 Archiviazione temporale nel livello gold

Il livello **gold** costituisce l'ultimo layer, ossia lo strato di persistenza progettato per servire con bassa latenza le richieste della dashboard web e delle API REST (ad esempio quando l'utente seleziona un paziente o cambia la finestra temporale dei grafici). Nel progettare quest'ultimo livello sono state confrontate tre opzioni architetturali:

- **Operare solo sui file Parquet:** i file Parquet sono ottimi per leggere o salvare un intero intervento dall'inizio alla fine, come avviene nel livello silver, ma non si rivelano adatti a fare da motore diretto per un'applicazione web. Ad ogni richiesta della dashboard il server dovrebbe infatti aprire i singoli file da disco, caricarli in memoria per calcolare le medie temporali e gestire contemporaneamente il salvataggio delle nuove anomalie rilevate, operazione scomoda su file pensati per essere scritti una volta sola.
- **Usare un database tradizionale come una tabella SQL o una collection MongoDB standard:** salvare ogni secondo come una riga o un documento indipendente risolverebbe il problema delle interrogazioni web, ma su $5,4$ milioni di secondi comporterebbe un forte spreco di spazio e di memoria. In una collection MongoDB normale, infatti, i nomi dei campi e i dati fissi del paziente (come `case_id`, `department`, età e sesso) verrebbero riscritti da capo in ciascuno dei milioni di documenti.
- **Adottare le collezioni temporali native (*time series*) di MongoDB 8.0:** questa soluzione, pensata specificamente per le serie storiche dei sensori, unisce la rapidità di interrogazione di un database con una compressione simile a quella dei file colonnari.

La scelta è ricaduta su quest'ultima soluzione, implementata nello script `src/gold/load_mongo.py` mediante la collezione `vital_signals`. Dichiarando la collezione come serie temporale nella funzione `ensure_timeseries_collections(db)`, il motore di archiviazione WiredTiger non memorizza un documento fisico indipendente per ogni secondo, ma raccoglie automaticamente le misure contigue appartenenti allo stesso paziente all'interno di contenitori compressi denominati **bucket**, gestiti nella collezione interna `system.buckets.vital_signals`. La configurazione poggia su tre parametri: `timeField: "timestamp"`, che definisce l'asse temporale di ordinamento; `metaField: "metadata"`, che indica i dati fissi del paziente (`case_id`, `sensor_name`, `department`, `age` e `sex`) usati per raggruppare i blocchi e creare un indice composto su metadati e tempo; e `granularity: "seconds"`, che dice a MongoDB che i dati arrivano al ritmo dei secondi ($1$ Hz), così da dimensionare ogni bucket fino a mille campioni o fino a un'ora di registrazione.

All'interno di ciascun bucket, MongoDB riorganizza i dati **per colonna** anziché per riga, mettendo cioè in fila tutti gli orari, tutti i valori di frequenza cardiaca, tutte le saturazioni, etc. Con questa disposizione, sono stati applicati tre meccanismi che riducono drasticamente lo spazio occupato su disco:

1. **Compressione degli orari:** salvare una data completa per ogni riga richiederebbe $64$ bit a secondo. Poiché però i campioni del monitor `Solar8000` arrivano a distanza costante di un secondo l'uno dall'altro: invece di memorizzare una data completa per ogni riga, il motore può quindi registrare soltanto la **differenza rispetto al passo precedente**, che è quasi sempre zero e occupa una frazione minima di spazio; a MongoDB basterà quindi memorizzare **un singolo bit** per indicare che il ritmo è rimasto invariato.
2. **Compressione dei valori numerici (differenze binarie e *Zstandard*):** i parametri vitali variano molto gradualmente da un secondo all'altro (e la pressione arteriosa, propagata con LOCF, resta addirittura costante fino a $300$ secondi consecutivi). Invece di salvare per intero ogni numero decimale a $64$ bit, MongoDB memorizza solo i bit che cambiano rispetto al secondo precedente (tramite confronto binario XOR) e comprime poi l'intero blocco con l'algoritmo **Zstandard**, che riduce a pochi byte le lunghe sequenze di valori uguali o molto vicini.
3. **Omissione dei valori mancanti in BSON:** mentre una tabella Parquet ha colonne fisse e deve comunque usare dei bit di controllo per segnalare che una cella è vuota (`NaN`), nei documenti BSON del livello gold un campo assente può semplicemente non essere scritto. Nella funzione `build_records`, quando un parametro vitale risulta `NaN` in un determinato secondo, la chiave corrispondente viene omessa dal dizionario `metrics`, facendo sì che i valori mancanti occupino **zero byte fisici** su disco.

## 3.2 Analisi sperimentale di spazio su disco, latenza e valori mancanti

Per valutare quantitativamente il comportamento dei tre livelli sulla coorte di $500$ pazienti è stato realizzato il modulo `src/analysis/benchmark_etl.py`, che salva le misurazioni nel file `data/benchmark_report.json` associandovi gli identificativi `run_id` e `pipeline_version` (`"1.1.0"`). Per garantire un confronto omogeneo tra file system e database, lo script interroga preliminarmente il catalogo `registry` prelevando la lista dei **500 casi con stato** `committed` ($5.402.386$ secondi attivi) e calcola le dimensioni su disco filtrando esattamente i medesimi cinquecento identificativi nei livelli bronze e silver. I risultati sperimentali sono riportati nella Tabella 3.1 e rappresentati graficamente nella Figura 3.1.

| Metrica sperimentale | Bronze (Parquet, 6 col.) | Silver (Parquet, 18 col.) | Gold (MongoDB 8.0) |
|---|---|---|---|
| Occupazione su disco (500 casi) | 33,95 MB | 44,52 MB (+31,1% con flag) | **15,60 MB** (-54,1% vs bronze / -65,0% vs silver) |
| Indici B-Tree (`totalIndexSize`) | non applicabile | non applicabile | +2,56 MB (totale `collStats`: 18,16 MB) |
| Latenza query singolo caso | 26,95 ms (Parquet) | 26,95 ms (Parquet) | **12,02 ms** (indice su `metaField`) |
| Fattore di velocizzazione | 1,0× (riferimento) | 1,0× | **2,2× più veloce** |
| Valori mancanti (tasso di null) | 66,3% | 26,3% (dopo LOCF) | 26,3% logico / **0 byte fisici** |

**Tabella 3.1:** Confronto sperimentale di spazio su disco, latenza di interrogazione e valori mancanti sui 500 casi clinici.

L'osservazione congiunta della Tabella 3.1 e della Figura 3.1 mette in luce la diversa funzione svolta dai livelli silver e gold. Nel passaggio dal livello bronze al livello silver il volume complessivo su disco cresce da **33,95 MB** a **44,52 MB** (**+31,1%**): questo incremento deriva dall'espansione dello schema da $6$ a $18$ colonne e dal riempimento di milioni di celle vuote con i valori di pressione propagati tramite LOCF, che riduce i valori mancanti dal $66,3\%$ al $26,3\%$. Quando i medesimi dati vengono trasferiti nella collezione temporale del livello gold, la compressione per bucket di MongoDB, usando le tecniche descritte precedentemente, riduce lo spazio fisico dei dati (`storageSize`) a **15,60 MB**, segnando un risparmio del **65,0%** rispetto a silver e del **54,1%**

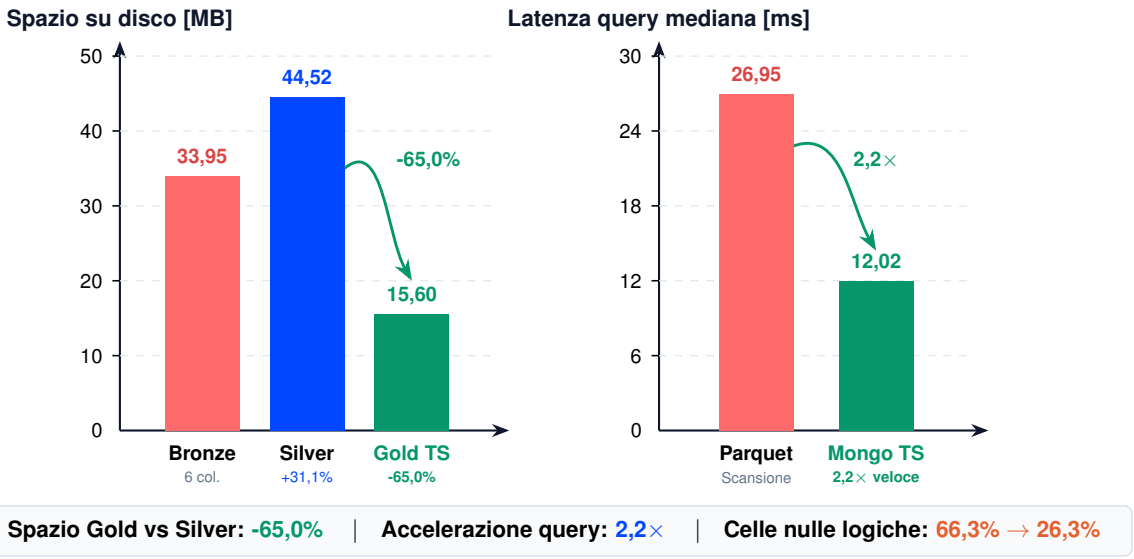



**Figura 3.1:** Confronto grafico dell'occupazione su disco, dei tempi di risposta e degli indicatori sintetici sui 500 interventi chirurgici.

rispetto a bronze. Nella tabella il peso dei dati (**15,60 MB**) è tenuto distinto da quello degli indici B-Tree (`totalIndexSize`, pari a **2,56 MB**), strutture che MongoDB mantiene per trovare più rapidamente le misure associate a un caso e a un intervallo temporale, per confrontare alla pari i dati compressi di MongoDB con i file Parquet, che nativamente sono privi di indici. Si può notare come anche sommando dati e indici (**18,16 MB** complessivi), il database occupa comunque meno della metà rispetto ai file su disco.

Un chiarimento importante riguarda l'ultima riga della Tabella 3.1, dove per il livello gold distinguiamo tra **26,3% di valori mancanti logici** e **0 byte fisici** occupati. Come visto nella Sezione 3.1, quando in un certo secondo manca la misura di un sensore, la funzione `build_records` non scrive affatto la relativa chiave nel dizionario `metrics` (non salva cioè `campo: null`). Di conseguenza, se cercassimo banalmente i campi con valore `null` dentro il database troveremmo zero occorrenze, come se non mancasse mai nessun dato. Per calcolare in modo corretto la percentuale di misure assenti anche nel livello gold, lo script `benchmark_etl.py` confronta il numero totale di misure che ci aspetteremmo se tutti i sensori fossero sempre presenti (documenti $\times$ 5) con il numero di volte in cui ciascuno dei cinque parametri compare effettivamente nei documenti, contato tramite una query con l'operatore `$exists: True`. La differenza restituisce esattamente il **26,3%**, lo stesso valore misurato sui file Parquet del livello silver: questo conferma che la trasformazione in documenti BSON mantiene intatto il contenuto informativo dei dati, pur azzerando lo spazio su disco dei campioni mancanti.

Per quanto riguarda le prestazioni di lettura, il benchmark confronta a parità di operazione il tempo necessario per calcolare la frequenza cardiaca media dell'intervento `case_id = 1`. Dopo un ciclo preliminare di riscaldamento della cache, la stessa operazione viene ripetuta 15 volte consecutivamente, e come risultato si considererà la mediana dei tempi, cioè il valore centrale delle misurazioni. Come si nota dalla Figura 3.1, la lettura e l'aggregazione del file Parquet partizionato richiedono **26,95 ms**, mentre la pipeline di aggregazione MongoDB composta dagli stadi di matching `$match` del caso su `metadata.case_id`, e di calcolo della media del gruppo `$group` richiede **12,02 ms**, risultando **2,2 volte più veloce** grazie all'accesso diretto ai soli bucket del paziente richiesto tramite l'indice sul `metaField`.

## 3.3 Architettura di data governance e gestione transazionale

Per garantire che i dati esposti nel livello gold siano completi, conformi allo schema atteso e ricollegabili ai file locali da cui derivano, è stato progettato un sistema di **data governance** applicativo che cerca di emulare quelli che sono i meccanismi essenziali di un **data lakehouse** direttamente su MongoDB 8.0 e Python, senza introdurre dipendenze da motori distribuiti come Apache Spark, Delta Lake o Apache Iceberg. La Tabella 3.2 sintetizza come ciascuna garanzia tipica di un lakehouse industriale sia stata tradotta, nei limiti delle possibilità, nella nostra architettura.

| **Requisito di governance** | **Implementazione su MongoDB e Python** | **Differenza rispetto a Delta Lake / Iceberg** |
|---|---|---|
| Registro delle transazioni e isolamento dei lettori | Collezione `registry` con stati `loading` → `committed` / `failed` e filtro `{"status": "committed"}` su tutte le letture. | Coordinamento applicativo in due fasi anziché transazione atomica nativa del motore di storage, assistito da `recover_incomplete_loads`. |
| Validazione dello schema e contratto dati | Validatore nativo `$jsonSchema` su `registry` e controllo preventivo `validate_records(records)` prima di `insert_many` su `vital_signals`. | MongoDB 8.0 non supporta `$jsonSchema` sulle collezioni temporali, rendendo necessario il controllo lato Python per `vital_signals`. |
| Catalogo dei metadati fisiologici | Dizionario `TRACK_SPECS` in `src/config.py` sincronizzato nella collezione `track_metadata` da `seed_track_metadata(db)`. | Sincronizzazione automatica ad ogni esecuzione di `load_mongo.py` senza necessità di un metastore esterno separato. |
| Tracciabilità delle esecuzioni | Identificativi `run_id` e `pipeline_version` condivisi nei quattro file JSON e nel `registry`, con storico in `quality_history/`. | Tracciabilità mantenuta a livello di caso nel catalogo `registry` per non appesantire i singoli documenti temporali. |

**Tabella 3.2:** Confronto tra i meccanismi di un data lakehouse industriale e l'implementazione realizzata su MongoDB 8.0.

### 3.3.1 Catalogo dei casi, contratti di schema e metadati delle tracce

Il fulcro dell'architettura di controllo è costituito dalla collezione `registry`, che mantiene un documento di stato per ciascuno dei $500$ casi clinici del livello gold. Ogni voce generata dalla funzione `build_registry_doc` in `src/gold/load_mongo.py` memorizza l'identificativo `case_id`, la versione dello schema (`schema_version: "1.0"`), lo stato corrente del caricamento (`status`, che può assumere i valori `loading`, `committed` oppure `failed`), il conteggio esatto dei secondi inseriti (`record_count`), i riferimenti `run_id` e `pipeline_version` e il sotto-documento `provenance`, nel quale sono registrati l'istante UTC di caricamento, i parametri di pulizia applicati. Tali parametri sono tre:

1. `max_gap_seconds: 5.0`: indica la durata massima di un buco nei segnali continui, che viene poi colmato tramite interpolazione lineare;
2. `nibp_staleness_limit_s: 300.0`: indica il tempo massimo per cui l'ultima misura del bracciale viene mantenuta costante con il metodo *LOCF* prima di essere considerata non più rappresentativa;

3. `sampling_interval_s: 1.0`: indica che il campionamento è avvenuto a intervalli di 1 secondo.

A livello di database, lo script di inizializzazione `mongo/registry_schema.js` impone sulla collezione `registry` un vincolo formale tramite l'operatore nativo `$jsonSchema` configurato con `validationLevel: "strict"` e `validationAction: "error"`, come mostrato nel Listato 3.1. In questo modo il motore di MongoDB rifiuta automaticamente qualsiasi inserimento o aggiornamento che risulti privo degli attributi obbligatori `case_id`, `schema_version` e `provenance` o che presenti tipi di dato differenti da quelli dichiarati.

```
// Schema definito in mongo/registry_schema.js
db.runCommand({
    collMod: "registry",
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["case_id", "schema_version", "provenance"],
            properties: {
                case_id: { bsonType: ["int", "long"], description: "ID caso VitalDB" },
                schema_version: { bsonType: "string", description: "Versione schema" },
                record_count: { bsonType: ["int", "long"], description: "Secondi caricati" },
                provenance: {
                    bsonType: "object",
                    required: ["step", "timestamp"],
                    properties: {
                        step: { bsonType: "string" },
                        timestamp: { bsonType: "date" }
                    }
                }
            }
        }
    },
    validationLevel: "strict",
    validationAction: "error"
});
```

**Listato 3.1:** Regola di validazione `$jsonSchema` applicata alla collezione `registry` in `mongo/registry_schema.js`.

Oltre alla collezione `registry` dedicata ai pazienti, nello stesso database MongoDB è presente una seconda collezione di supporto chiamata `track_metadata`, che funge da catalogo descrittivo dei sensori: contiene cinque documenti, uno per ciascuna traccia del monitor `Solar8000`, specificando per ognuno l'unità di misura (`bpm`, `%`, `mmHg`), il significato clinico e l'intervallo di valori fisiologici validi per ogni traccia specifica (`expected_range`). La prima creazione del catalogo è affidata allo script `mongo/registry_schema.js`, montato nella cartella `/docker-entrypoint-initdb.d/` del container Docker di MongoDB. Poiché Docker esegue gli script presenti in `/docker-entrypoint-initdb.d/` soltanto al primo avvio assoluto, cioè quando il volume dei dati è ancora vuoto, una modifica successiva alle soglie dentro `config.TRACK_SPECS` rischierebbe di non riflettersi nel database; per ovviare a questo problema di staticità, e quindi per evitare qualsiasi disallineamento, senza dover cancellare e ricreare il volume Docker ad ogni modifica, all'inizio di ogni esecuzione di `process_silver_to_gold` viene invocata la funzione `seed_`

`track_metadata(db)`, che legge i valori aggiornati da `config.TRACK_SPECS` ed esegue su ciascuna delle cinque tracce un comando `update_one(..., upsert=True)`, creando il documento se non esiste ancora oppure aggiornandolo sul posto con le nuove soglie.

Proprio l'applicazione di queste soglie ai tracciati dei pazienti collega il catalogo dei sensori alla misurazione della qualità dei dati: mentre infatti la collezione `registry` tiene traccia dei pazienti e della durata complessiva degli interventi, e `track_metadata` definisce le regole generali dei cinque canali, per valutare l'affidabilità di un intervento serve sapere come si è comportato ciascun sensore su quel singolo paziente. Questa informazione è però indispensabile, perché la qualità del segnale varia moltissimo da un paziente all'altro: una misura **per singolo intervento**, rispetto a una statistica globale, permette di capire quanto ci si possa fidare dei risultati ottenuti su quel paziente, di individuare i casi da escludere o da interpretare con cautela, e di rendere ogni scelta di pulizia verificabile anche a posteriori. Ecco perchè nel livello Silver, mentre bonifica ciascun caso, ne calcola anche un resoconto della sua qualità e lo salva nel file `data/quality_report.json`, in un elemento dedicato a ogni caso, all'interno del campo `sensor_metrics`. Per ciascun canale, il resoconto si fonda su una classificazione completa dei secondi registrati: dopo il trattamento dei dati nella fase Silver, ogni riga del file silver (`rows_cleaned`) viene assegnata a una sola di tre categorie, ovvero valore misurato direttamente dal sensore (`observed`), valore ricostruito dalla pulizia tramite propagazione *LOCF* o interpolazione (`imputed`) e valore rimasto vuoto (`missing`), cioè `NaN`. In questo modo, su ciascuno dei cinque canali, ogni singolo secondo attivo viene classificato in una sola di tre condizioni possibili, garantendo la perfetta uguaglianza contabile:

$$\texttt{observed} + \texttt{imputed} + \texttt{missing} = \texttt{rows_cleaned}$$

I tre termini descrivono con precisione l'origine di ciascun valore presente nella serie temporale finale:

- **`observed`:** i secondi in cui la misura è stata acquisita direttamente dallo strumento in sala operatoria, senza alcun intervento correttivo da parte del software;
- **`imputed`:** i secondi originariamente vuoti (o mascherati perché fuori scala) che sono stati ricostruiti dalla pipeline, conteggiati sommando i rispettivi flag booleani (`HR_imputed`, `SPO2_imputed`, `SBP_imputed`, `DBP_imputed` o `MBP_imputed`) generati tramite interpolazione lineare o propagazione LOCF;
- **`missing`:** i secondi rimasti a `NaN` anche al termine della bonifica, corrispondenti a disconnessioni dei sensori continui superiori ai $5$ secondi o a intervalli tra due gonfiaggi del bracciale pressorio superiori ai $300$ secondi.

A partire da questo bilancio, lo script calcola per ciascuna traccia tre indicatori percentuali complementari, ciascuno con una specifica finalità:

1. $\texttt{observed_pct} = \frac{\texttt{observed}}{\texttt{rows_cleaned}} \times 100$: quantifica la percentuale di letture reali fornite dallo strumento, permettendo di valutare l'effettiva fedeltà biologica del tracciato prima della nostra elaborazione;
2. $\texttt{coverage_pct} = \frac{\texttt{observed}+\texttt{imputed}}{\texttt{rows_cleaned}} \times 100$: misura la copertura complessiva messa a disposizione dei modelli di Machine Learning rispetto ai secondi attivi dell'intervento, evidenziando il guadagno ottenuto grazie alla fase di imputazione;
3. $\texttt{temporal_coverage_bronze_pct} = \frac{\texttt{observed}+\texttt{imputed}}{\texttt{rows_original}} \times 100$: rapporta i secondi utilizzabili alla durata totale grezza del file bronze originale. Questo terzo indice funge da controllo di sicurezza: evita infatti che un intervento in cui i sensori sono rimasti scollegati per quasi tutta l'operazione appaia coperto al $100\%$ solo perché i periodi inattivi sono stati rimossi dalla tabella pulita.

### 3.3.2 Protocollo di caricamento a due fasi e recupero automatico

L'implementazione del caricamento verso il livello gold ha richiesto di affrontare due restrizioni architetturali proprie delle collezioni temporali di MongoDB 8.0: da un lato, il motore del database non consente di inserire documenti in una collezione time series all'interno di una transazione multi-documento ACID, restituendo l'errore nativo 263 (`OperationNotSupportedInTransaction`); dall'altro, non permette di associare un validatore `$jsonSchema` lato server alla collezione `vital_signals`. Queste due restrizioni rendono il caricamento di un paziente un'operazione delicata, perché ciascun caso viene scritto con una singola `insert_many` di migliaia di documenti, uno per ogni secondo di registrazione. Senza una transazione l'operazione non è **atomica**: se il processo si interrompe a metà, per un errore di rete, un crash del container o un documento rifiutato, nella collezione resta una porzione parziale della serie senza un meccanismo nativo che la annulli. Un lettore, come il modulo di rilevamento o la dashboard, la tratterebbe come completa senza alcun segnale di anomalia, con conseguenze dirette sull'autoencoder, sulla media mobile della frequenza cardiaca e sulle statistiche per paziente che verrebbero calcolate su dati incompleti, producendo risultati errati, sebbene plausibili a prima vista. A ciò si aggiunge il problema della **ripetibilità**: rieseguendo la pipeline da capo, un caso già caricato deve essere sostituito e non duplicato, ma senza transazioni cancellazione e inserimento non possono essere eseguiti come un'unica operazione indivisibile. L'assenza di un validatore lato server comporta infine che il database accetterebbe silenziosamente documenti malformati, che una volta compressi nei bucket sarebbero difficili da individuare e da correggere. Poiché né l'isolamento dei lettori né il controllo dei dati possono essere delegati al motore, entrambi sono stati portati nel codice applicativo. Infatti, per garantire comunque l'integrità dei dati e l'isolamento dei lettori senza ricorrere a transazioni native, in `src/gold/load_mongo.py` è stato sviluppato un protocollo applicativo a due fasi abbinato a una validazione preventiva in memoria. Prima di avviare qualsiasi scrittura sul database, la funzione `validate_records(records)` ispeziona tutti i documenti generati per il paziente verificando che `timestamp` sia un oggetto `datetime` valido, che `metadata.case_id` sia un intero nativo, che `metadata.sensor_name` sia una stringa non vuota e che tutti i valori contenuti in `metrics` siano numeri finiti privi di `NaN`, sollevando un'eccezione `ValueError` in caso di violazione.

Superato il controllo preventivo in memoria, la funzione `load_case_to_mongo` avvia il protocollo di caricamento a due fasi, progettato specificamente per sopperire all'assenza di un meccanismo di *rollback* automatico nativo nel motore temporale di MongoDB. Nella prima fase, lo script registra nel catalogo `registry` lo stato transitorio `status: "loading"` con `record_count: 0` e ripulisce preventivamente la collezione temporale da eventuali punti pregressi del paziente tramite `db['vital_signals'].delete_many({"metadata.case_id": int(case_id)})`. Nella seconda fase viene quindi tentato l'inserimento massivo dei documenti tramite `db['vital_signals'].insert_many(records)`:

- **In caso di successo:** il documento nel `registry` viene promosso allo stato definitivo `status: "committed"`, memorizzando il conteggio effettivo dei secondi caricati e i metadati completi di provenienza;
- **In caso di errore gestito (`PyMongoError`):** non potendo invocare un comando di *rollback* a livello di database, lo script esegue una **transazione di compensazione applicativa** nel blocco di gestione delle eccezioni, eliminando immediatamente i punti parziali appena scritti in `vital_signals` e marcando il caso come `status: "failed"` con il dettaglio dell'errore nel campo `last_error`.

L'assenza di un gestore transazionale nativo impone però di tutelarsi anche contro lo scenario più critico: un'interruzione brusca del processo Python o del container avvenuta *durante* la scrittura dei dati, che impedirebbe persino l'esecuzione del codice di compensazione. In un database tradizionale con supporto ACID il motore annullerebbe le scritture incomplete al riavvio; in MongoDB Time Series, invece, i documenti parziali rimarrebbero abbandonati sul disco. Per

risolvere alla radice questo problema, l'architettura implementa una duplice tecnica:

1. **Isolamento dei lettori:** tutti i moduli a valle che vedremo più avanti (rotte REST dell'API, grafici della dashboard web e modelli di Machine Learning) interrogano il database applicando tassativamente il filtro `{"status": "committed"}`; in questo modo un paziente rimasto "congelato" nello stato `"loading"` o `"failed"` risulta del tutto invisibile a qualsiasi utilizzatore;
2. **Autoguarigione e recupero al riavvio:** all'inizio di ogni successiva esecuzione della pipeline, la funzione `recover_incomplete_loads(db, active_case_ids)` scansiona il catalogo `registry`, individua tutti i casi rimasti in uno stato diverso da `committed` e ne rimuove i campioni orfani da `vital_signals`, eliminando inoltre eventuali pazienti non più inclusi nel perimetro silver corrente.

Nel complesso, il protocollo a due fasi non riproduce le garanzie ACID di una transazione nativa, ma ne ottiene gli effetti pratici più importanti. Il marcatore di stato nel `registry` impedisce che una serie parziale venga scambiata per completa, la compensazione applicativa e il recupero all'avvio eliminano i residui di caricamenti interrotti, e la cancellazione preventiva dei punti del paziente rende il caricamento **ripetibile**: rieseguire la pipeline sostituisce i dati invece di duplicarli. Il costo è una finestra temporale in cui il caso risulta in caricamento e non è ancora consultabile, mentre il vantaggio è che ogni caso visibile ai moduli a valle è, per costruzione, completo e validato.

## 3.4 Pipeline di aggregazione e ottimizzazione delle interrogazioni

Una volta garantite l'integrità formale e la tracciabilità dei dati tramite i meccanismi di governance, l'ultimo requisito fondamentale del livello gold riguarda la sua **servibilità**, ovvero la capacità di rendere fruibili queste grandi moli di serie temporali con tempi di risposta immediati verso la dashboard web e le rotte dell'interfaccia REST. In particolare, per mantenere fluida l'interazione sulla dashboard web anche quando si esplorano interventi chirurgici della durata di molte ore, le operazioni di riduzione temporale e di recupero dei metadati sono state delegate direttamente al motore di aggregazione di MongoDB nel modulo `src/analysis/queries.py`. Per esempio, un intervento di sei ore a $1$ Hz equivale a 21.600 istanti, e a ogni istante corrisponde un documento con i valori disponibili delle diverse tracce, quindi più di ventimila documenti: trasferire sistematicamente tutti quei punti via HTTP verso il browser appesantirebbe il disegno dei grafici quando l'utente desidera osservare l'andamento complessivo dell'operazione.

La funzione `downsample_case(db, case_id, window_seconds)` risolve il problema eseguendo sul database la pipeline a quattro stadi riportata nel Listato 3.2. Il primo stadio `$match` seleziona i bucket del singolo paziente sfruttando l'indice su `metadata.case_id`; il secondo stadio `$group` applica l'operatore nativo `$dateTrunc` per discretizzare l'asse temporale in finestre regolari di ampiezza configurabile ($30$, $60$ oppure $300$ secondi) calcolando con `$avg` il valore medio dei cinque parametri vitali in ciascun intervallo; gli stadi finali `$project` e `$sort` riformattano l'istante iniziale della finestra nel campo `window_start` e restituiscono la sequenza ordinata cronologicamente.

```
// Pipeline eseguita da downsample_case() in src/analysis/queries.py
pipeline = [
    {"$match": {"metadata.case_id": case_id}},
    {
        "$group": {
            "_id": {
                "$dateTrunc": {
                    "date": "$timestamp",
                    "unit": "second",
                    "binSize": window_seconds
                }
            },
            "Solar8000_HR":         {"$avg": "$metrics.Solar8000_HR"},
            "Solar8000_PLETH_SPO2": {"$avg": "$metrics.Solar8000_PLETH_SPO2"},
            "Solar8000_NIBP_SBP":   {"$avg": "$metrics.Solar8000_NIBP_SBP"},
            "Solar8000_NIBP_DBP":   {"$avg": "$metrics.Solar8000_NIBP_DBP"},
            "Solar8000_NIBP_MBP":   {"$avg": "$metrics.Solar8000_NIBP_MBP"},
        }
    },
    {"$project": {"_id": 0, "window_start": "$_id", ...}},
    {"$sort": {"window_start": 1}}
]
```

**Listato 3.2:** Pipeline di aggregazione con operatore `$dateTrunc` per la riduzione temporale in `src/analysis/queries.py`.

Un'ulteriore ottimizzazione riguarda il popolamento della barra laterale dei pazienti nella dashboard tramite la rotta `GET /cases`, che deve mostrare per tutti i $500$ interventi l'identificativo, il conteggio dei punti, il reparto chirurgico, l'età e il sesso. Poiché i documenti della collezione `registry` contengono `case_id` e `record_count` ma non i dati anagrafici, un'implementazione ingenua eseguirebbe prima una query sul catalogo e poi $500$ interrogazioni `find_one` distinte su `vital_signals` (schema $N + 1$ pari a $501$ chiamate di rete). Nel file `api/main.py` il caricamento è stato invece ricondotto a due sole interrogazioni complessive: una lettura dei casi `committed` da `registry` tramite `list_loaded_cases(db)` e un'unica pipeline di aggregazione su `vital_signals` con stadio `$group` su `$metadata.case_id` e operatore `$first` sui campi `department`, `age` e `sex`, unendo infine i due dizionari in memoria centrale.

Capitolo 4

# Modelli di anomaly detection

## 4.1 Impostazione dell'approccio ibrido

Nel monitoraggio intraoperatorio un paziente può andare incontro a instabilità emodinamica attraverso dinamiche molto eterogenee: in alcuni casi si assiste alla variazione acuta e repentina di un singolo parametro vitale, come un crollo improvviso della saturazione o della pressione arteriosa, mentre in altre situazioni il deterioramento si manifesta con oscillazioni più lente e distribuite contemporaneamente su più segnali. Nella progettazione del modulo di rilevamento delle anomalie, implementato in `src/detection/detector.py`, il primo vincolo strutturale è rappresentato dall'assenza nel dataset VitalDB di annotazioni cliniche secondo per secondo certificate da un medico anestesista, circostanza che esclude la possibilità di addestrare classificatori supervisionati tradizionali.

D'altra parte, l'impiego esclusivo di regole mediche a soglia fissa oppure di soli algoritmi di apprendimento non supervisionato presenterebbe limiti opposti. Le regole cliniche deterministiche possiedono un significato fisiologico immediato e trasparente per il personale sanitario, ma osservano una o due variabili per volta in modo statico e si attivano soltanto quando la soglia critica è già stata oltrepassata. Al contrario, i modelli di **machine learning** non supervisionato sono in grado di ispezionare congiuntamente tutte le variabili biometriche e le loro derivate temporali senza richiedere etichette di addestramento, ma producono un punteggio di atipicità statistica che da solo non distingue un'emergenza clinica da una variazione benigna del tracciato.

Per coniugare interpretabilità medica e capacità di analisi multivariata, è stata quindi adottata una strategia ibrida articolata su quattro metodi complementari: due regole cliniche deterministiche dominio-specifiche, basate sullo **shock index** ($\mathrm{SI} > 0,9$) e sull'**ipotensione severa concomitante a ipossiemia** ($\mathrm{MBP} < 65$ mmHg $\wedge\ \mathrm{SpO}_2 < 90\%$), affiancate da due modelli di apprendimento non supervisionato operanti su uno spazio a otto dimensioni, ovvero l'algoritmo ad alberi **isolation forest** e una rete neurale ricorrente **autoencoder LSTM** realizzata in PyTorch. L'abbinamento tra un modello basato sul partizionamento casuale dello spazio delle feature e un'architettura ricorrente su finestre scorrevoli di $30$ secondi consente di confrontare un rilevatore puntuale molto rapido con un modello capace di catturare le dipendenze sequenziali nel tempo.

## 4.2 Estrazione dal livello gold e formulazione delle regole cliniche

Prima di applicare le regole e i modelli, la funzione `load_from_gold(db, case_ids)` in `src/detection/detector.py` preleva le serie temporali dal livello gold adottando tre accorgimenti essenziali per la tenuta della memoria e la correttezza clinica. In primo luogo, la funzione interroga il catalogo `registry` selezionando unicamente i pazienti con stato `committed` e, quando il numero di interventi richiesti supera le $25$ unità (come nell'elaborazione dell'intera coorte da $500$ casi), suddivide la lettura da `vital_signals` in blocchi da `batch_size = 25` pazienti tramite la funzione ausiliaria `_fetch_gold_batch`, ordinando ciascun blocco per `("metadata.case_id", 1)` e `("timestamp", 1)` così da evitare picchi di allocazione nel cursore PyMongo su $5,4$ milioni di

documenti.

In secondo luogo, la funzione esegue il mascheramento attivo degli artefatti strumentali: poiché il livello silver mantiene i valori grezzi fuori scala per finalità di tracciabilità limitandosi a segnalarli nei flag booleani `quality_flags.*_outlier`, `load_from_gold` legge ciascuno dei cinque flag e sostituisce in memoria i campioni corrispondenti con `np.nan` prima di calcolare qualsiasi indice, impedendo che un disturbo da elettrobisturi (HR $= 320$ bpm) generi falsi allarmi a cascata. Infine, viene introdotto un vincolo rigoroso sulla freschezza della misura pressoria per le regole cliniche. Mentre nel livello silver le letture intermittenti del bracciale vengono propagate in avanti fino a $300$ secondi per fornire una matrice densa ai modelli multivariati, sul piano medico non sarebbe corretto mantenere attivo un allarme acuto basandosi su una pressione rilevata quattro o cinque minuti prima. Entrambe le regole cliniche leggono quindi il contatore `quality_flags.nibp_age_s` e applicano la soglia configurata in `config.NIBP_MAX_CLINICAL_AGE_S = 120.0` secondi, valutando il dato pressorio solo se acquisito negli ultimi due minuti.

La prima regola deterministica calcola lo **shock index** (SI), un indicatore emodinamico adimensionale impiegato in anestesia e rianimazione per intercettare stati di shock ipovolemico o emorragico incipiente, definito dal rapporto tra la frequenza cardiaca e la pressione arteriosa sistolica:

$$\text{SI} = \frac{\text{Solar8000_HR}}{\text{Solar8000_NIBP_SBP}}$$

In condizioni fisiologiche stabili il rapporto si colloca tra $0,5$ e $0,7$; in presenza di ipovolemia o marcata vasodilatazione da farmaci anestetici, l'organismo attiva una tachicardia compensatoria mentre la pressione sistolica flette, spingendo l'indice verso l'alto. La funzione `compute_shock_index(df, threshold=0.9, max_nibp_age_s=None)`, dopo aver convertito a `np.nan` eventuali valori non positivi del denominatore, contrassegna come anomalo ogni secondo che soddisfa la condizione:

$$\text{SI} > 0,9 \quad \wedge \quad \texttt{nibp_age_s} \leq 120 \text{ s}$$

La seconda regola clinica, implementata nella funzione `compute_severe_hypotension(df, mbp_threshold=65.0, spo2_threshold=90.0, max_nibp_age_s=None)`, individua gli episodi critici in cui un'ipoperfusione arteriosa severa si manifesta simultaneamente a una desaturazione periferica d'ossigeno:

$$\text{Solar8000_NIBP_MBP} < 65 \text{ mmHg} \wedge \text{Solar8000_PLETH_SPO2} < 90\% \wedge \texttt{nibp_age_s} \leq 120 \text{ s}$$

Il contemporaneo superamento verso il basso della soglia di perfusione d'organo ($65$ mmHg) e della soglia ipossiemica ($90\%$) su una misura pressoria fresca identifica infatti una condizione di immediata sofferenza ipossica-ischemica dei tessuti.

## 4.3 Ingegnerizzazione delle feature temporali e imputazione locale

Se i modelli di apprendimento automatico ricevessero in ingresso soltanto i cinque parametri biometrici istantanei, un algoritmo privo di memoria sequenziale come isolation forest tratterebbe ogni secondo come un punto indipendente, senza poter distinguere una frequenza cardiaca stabile a $95$ bpm da un battito che è appena salito bruscamente da $60$ a $95$ bpm in pochi istanti. Per incorporare nel vettore di stato anche la dinamica locale senza moltiplicare eccessivamente il numero di dimensioni, nella funzione `run_detection_pipeline` vengono calcolate tre feature derivate a partire dal segnale più continuo e affidabile della coorte, ovvero la frequenza cardiaca `Solar8000_HR` campionata a $1$ Hz. Indicando con $x_t$ il valore di frequenza cardiaca al secondo $t$ e adottando una finestra mobile retrospettiva di ampiezza $W = 5$ secondi (`rolling(5, min_periods=1)`), intervallo pari a circa $5$–$7$ battiti cardiaci ideale per catturare la variabilità istantanea

senza introdurre ritardi, lo script ricava la media mobile a cinque secondi `HR_rolling_mean` che smorza le fluttuazioni istantanee:

$$\mu_t = \frac{1}{W} \sum_{k=0}^{W-1} x_{t-k}$$

la deviazione standard mobile `HR_rolling_std` (inizializzata a `0.0` sul primo campione di ciascun intervento) che quantifica la variabilità a brevissimo termine del ritmo cardiaco:

$$\sigma_t = \sqrt{\frac{1}{W-1} \sum_{k=0}^{W-1} (x_{t-k} - \mu_t)^2}$$

e infine la differenza prima `HR_delta` (`.diff().fillna(0.0)`), che misura la velocità di variazione istantanea da un secondo al successivo:

$$\Delta x_t = x_t - x_{t-1}$$

Tutte e tre le trasformazioni vengono eseguite raggruppando la tabella per paziente tramite l'espressione `df.groupby("case_id", sort=False)["Solar8000_HR"]`, evitando qualsiasi contaminazione tra gli ultimi campioni di un intervento e i primi secondi del paziente successivo. Lo spazio delle variabili fornito ai modelli non supervisionati risulta così formato da **8 feature numeriche**: i cinque canali biometrici (`Solar8000_HR`, `Solar8000_PLETH_SPO2`, `Solar8000_NIBP_SBP`, `Solar8000_NIBP_DBP` e `Solar8000_NIBP_MBP`) uniti alle tre grandezze dinamiche (`HR_rolling_mean`, `HR_rolling_std` e `HR_delta`). La colonna `nibp_age_s` è stata invece deliberatamente esclusa dalle feature di addestramento: trattandosi di un contatore strumentale di anzianità della misura e non di un parametro fisiologico, se data in pasto agli algoritmi farebbe apprendere che un intervallo prolungato tra due gonfiaggi (ad esempio $280$ secondi) sia un'anomalia del paziente, confondendo la normale latenza meccanica del bracciale con un'emergenza clinica acuta.

Poiché né l'implementazione di `IsolationForest` in scikit-learn né i tensori di PyTorch ammettono valori `NaN` in ingresso, prima dell'addestramento è necessario completare le eventuali lacune residue (come i periodi in cui il bracciale pressorio è rimasto inattivo per oltre $300$ secondi o i campioni appena mascherati perché outlier). Durante lo sviluppo si è individuato e corretto un sottile difetto legato al concatenamento dei metodi in Pandas: scrivendo `df.groupby("case_id")[feat_cols].ffill().bfill()`, il primo `.ffill()` opera correttamente all'interno di ciascun paziente ma restituisce un `DataFrame` ordinario non più raggruppato, per cui il successivo `.bfill()` finiva per propagare all'indietro i valori del paziente seguente qualora i primi minuti di un intervento fossero privi di letture pressorie. La correzione, applicata in tutti i moduli di rilevamento e di valutazione e presidiata dal test unitario `test_imputation_no_bfill_leakage_across_cases`, riapplica esplicitamente il raggruppamento per `case_id` anche nella propagazione all'indietro:

```
df.groupby("case_id", sort=False)[feat_cols].ffill().groupby(df["case_id"], sort=False).bfill().fillna(0.0)
```

confinando rigorosamente ogni imputazione all'interno del singolo caso clinico.

## 4.4 Modelli non supervisionati: isolation forest e autoencoder LSTM

### 4.4.1 Partizionamento casuale nello spazio a otto dimensioni con isolation forest

Il primo modello di apprendimento non supervisionato, implementato nella funzione `detect_isolation_forest(df, contamination=0.05)`, impiega l'algoritmo **isolation forest** della libreria

scikit-learn (`random_state=42`). Invece di stimare la densità locale dei campioni normali, operazione computazionalmente onerosa su $5,4$ milioni di righe, l'algoritmo costruisce un insieme di alberi binari di isolamento selezionando ricorsivamente in modo casuale una delle otto feature e un valore di taglio compreso tra il minimo e il massimo. I campioni fisiologici regolari, addensati attorno al centro della distribuzione multivariata, richiedono numerose partizioni successive prima di essere isolati in una foglia; al contrario, i punti che presentano valori estremi o combinazioni inusuali tra le otto variabili (come un'elevata instabilità `HR_rolling_std` associata a una flessione della pressione media) vengono separati dal resto della coorte in pochissimi tagli vicino alla radice dell'albero, ottenendo una lunghezza media del cammino ridotta e un punteggio di anomalia elevato.

Impostando l'iperparametro `contamination=0.05` (così come la soglia al $95°$ percentile nel successivo modello neurale), l'algoritmo ordina tutti i secondi analizzati dal più regolare al più atipico ed etichetta come anomalo il $5\%$ dei campioni situati nella coda estrema della distribuzione. Quando la pipeline viene eseguita sull'intera coorte di $500$ pazienti, il taglio del $5\%$ viene applicato sulla distribuzione globale dei punteggi, facendo sì che gli interventi clinicamente più turbolenti concentrino una quota maggiore di segnalazioni rispetto ai pazienti emodinamicamente stabili.

### 4.4.2 Architettura ricorrente dell'autoencoder LSTM e gestione della memoria

Per intercettare anomalie che si sviluppano come sequenze temporali anomale anziché come singoli picchi istantanei, il secondo modello adotta un **autoencoder LSTM** realizzato in PyTorch, che analizza finestre scorrevoli di **30 secondi consecutivi** su tutte le otto feature standardizzate con `StandardScaler` (media nulla e varianza unitaria): l'orizzonte di $30$ secondi costituisce la classica finestra clinica in cui si manifestano transizioni di fase emodinamiche acute (come la reazione a farmaci o manovre chirurgiche). Come illustrato nella Figura 4.1 e nel Listato 4.1, la rete neurale `_LSTMAutoencoder` riceve in ingresso un tensore tridimensionale di forma $(B, 30, 8)$ e lo comprime attraverso un codificatore ricorrente `nn.LSTM(n_features=8, hidden_size=32, batch_first=True)` in un vettore di contesto latente $h$ a $32$ dimensioni; tale rappresentazione compressa viene replicata per tutti i $30$ passi temporali della finestra e inviata al decodificatore `nn.LSTM(32, 32, batch_first=True)`.

All'uscita del decodificatore è stato inserito uno strato di proiezione lineare `self.out = nn.Linear(32, 8)`. Questa scelta progettuale risolve un limite intrinseco delle celle LSTM standard, la cui uscita passa attraverso una funzione di attivazione tangente iperbolica ($\tanh$) limitata all'intervallo $[-1, +1]$: poiché le otto feature standardizzate in punteggio Z assumono regolarmente valori superiori a $\pm 2$ o $\pm 3$ durante le oscillazioni fisiologiche, l'assenza del proiettore lineare finale impedirebbe fisicamente alla rete di ricostruire i valori oltre l'unità.




**Figura 4.1:** Architettura della rete autoencoder LSTM a sette stadi con proiettore lineare e calcolo dell'errore massimo sui canali.

```
def _make_lstm_autoencoder_class():
    """Costruisce la classe LSTMAutoencoder come nn.Module solo se PyTorch e' disponibile."""
    if not _TORCH_AVAILABLE:
        return None

    class _LSTMAutoencoder(nn.Module):
        def __init__(self, n_features, hidden_size=32, n_layers=1):
            super().__init__()
            # Encoder LSTM: comprime la sequenza in un vettore latente
            self.encoder = nn.LSTM(n_features, hidden_size, n_layers, batch_first=True)
            # Decoder LSTM: decodifica il vettore latente nello spazio nascosto
            self.decoder = nn.LSTM(hidden_size, hidden_size, n_layers, batch_first=True)
            # Layer di proiezione lineare: mappa lo stato nascosto nello spazio continuo R^n
            self.out = nn.Linear(hidden_size, n_features)

        def forward(self, x):
            # x ha forma (batch, T, n_features)
            _, (hidden, _) = self.encoder(x)
            # Ripete il vettore latente per ogni timestep della sequenza
            context = hidden.permute(1, 0, 2).repeat(1, x.size(1), 1)
            output, _ = self.decoder(context)
            return self.out(output)  # dimensione: (batch, T, n_features)

    return _LSTMAutoencoder
```

**Listato 4.1:** Definizione della rete PyTorch `_LSTMAutoencoder` con proiettore lineare in `src/detection/detector.py`.

Passando dai primi collaudi su piccola scala all'intera coorte di **500 interventi** ($5.402.386$ secondi), la generazione di finestre scorrevoli di ampiezza $30$ e passo $1$ produce circa $5,39$ milioni di matrici di dimensione $(30, 8)$: materializzare tutte le finestre in un unico array tridimensionale tramite `np.concatenate` avrebbe richiesto oltre **5,17 GB** di memoria RAM in `float32`, causando l'arresto del container Docker per esaurimento della memoria. Nella classe `LSTMAutoencoderDetector` il problema è stato risolto combinando tre tecniche. In primo luogo, il metodo `_make_window_views_per_case` converte i dati normalizzati in `np.float32` e costruisce per ciascun paziente una vista virtuale senza copia in memoria tramite `np.lib.stride_tricks.sliding_window_view(vals, window_shape=(30, 8))`, che modifica unicamente i passi di indicizzazione (**stride**) sull'array bidimensionale esistente evitando anche di creare finestre a cavallo tra due pazienti diversi. In secondo luogo, quando il numero complessivo di finestre supera `max_train = 15000`, la fase di addestramento estrae dalle viste un sottocampione uniforme di $15.000$ finestre distribuite lungo l'intera coorte (`np.linspace(0, total_windows - 1, max_train, dtype=int)`), che occupa appena **14,4 MB** di RAM, addestrando la rete per $5$ epoche con ottimizzatore Adam (`lr=1e-3`). Infine, la fase di inferenza valuta comunque tutte le $5,39$ milioni di finestre scorrendo le viste virtuali a blocchi di `eval_batch_size = 4096` elementi dentro un contesto `torch.no_grad()`.

Anche la metrica con cui viene sintetizzato l'errore di ricostruzione di ciascuna finestra è stata oggetto di affinamento. Poiché le tre tracce pressorie rimangono costanti per lunghi tratti per effetto della propagazione LOCF, l'autoencoder impara a ricostruirle con errore prossimo allo zero; se si calcolasse la media aritmetica dell'errore quadratico su tutte e otto le colonne, il crollo improvviso di un singolo canale vitale come la saturazione $SpO_2$ verrebbe diviso per otto e mascherato dagli altri sette canali stabili. Per evitare questa diluizione, lo script calcola prima l'errore quadratico medio temporale sui $30$ secondi di ciascuna variabile (`mean_time_err = torch.mean(sq_err, dim=1)`) e seleziona poi il **massimo errore tra gli otto canali** (`batch_mse, _ = torch.max(mean_time_err, dim=1)`):

$$\mathcal{E}(X) = \max_{c \in \{1,\dots,8\}} \left( \frac{1}{30} \sum_{t=1}^{30} (X_{t,c} - \hat{X}_{t,c})^2 \right)$$

Il punteggio $\mathcal{E}(X)$ di ciascuna finestra viene attribuito al suo ultimo secondo (`case_idx[self.window_size - 1:]`) ed etichetta come anomali gli istanti che superano il $95°$ percentile della distribuzione degli errori.

Il confronto tra i due approcci non supervisionati riflette anche un chiaro compromesso operativo tra complessità ed efficienza computazionale. Mentre le regole cliniche deterministiche operano a costo $O(1)$ nell'ordine dei microsecondi e isolation forest completa l'inferenza spaziale di un intero intervento in meno di un secondo, l'autoencoder LSTM richiede il caricamento dei tensori e la propagazione ricorrente su PyTorch: pur risultando rapido grazie al batching a $4096$ campioni, esso presenta un costo di calcolo superiore. In un'architettura ospedaliera reale, questa gerarchia consente di impiegare le regole e il partizionamento ad alberi come guardiani a latenza zero direttamente a bordo monitor (edge streaming), riservando alla rete ricorrente l'analisi retrospettiva più approfondita sui pattern sequenziali.

## 4.5 Valutazione sperimentale sui 500 casi e persistenza degli allarmi

Per analizzare il comportamento dei due modelli non supervisionati sull'intera coorte di $500$ pazienti è stato predisposto il modulo `src/analysis/evaluate_models.py`, che salva i risultati in

`data/ml_evaluation.json`. In assenza di annotazioni mediche esterne, lo script costruisce un riferimento interno (**pseudo-ground truth** clinica) definendo come istanti positivi i secondi in cui si attiva almeno una delle due regole cliniche deterministiche (shock index $> 0,9$ oppure ipotensione severa MBP $< 65$ mmHg $\wedge$ $SpO_2 < 90\%$, entrambe vincolate a `nibp_age_s` $\leq 120$ s). Su un totale di **5.402.386 campioni** validati nel livello gold, le regole cliniche individuano **262.978 secondi positivi**, pari a una prevalenza (`base_rate_pct`) del **4,87%**.

Di fronte a una classe positiva pari al $4,87\%$ contro il $95,13\%$ di campioni stabili, l'accuratezza globale risulterebbe priva di valore informativo, poiché un classificatore banale che predicesse sempre la classe normale otterrebbe un'accuratezza del $95,13\%$ senza intercettare alcun evento critico. La valutazione utilizza quindi esclusivamente metriche specifiche per distribuzioni sbilanciate: **precision**, **recall**, **F1-score** sulla classe anomala e soprattutto l'area sotto la curva precisione-richiamo **PR-AUC** (`average_precision_score`), il cui valore di riferimento per un classificatore casuale coincide con la prevalenza della classe positiva ($0,0487$). I risultati sono riassunti nella Tabella 4.1.

| Modello valutato | Precision (anom.) | Recall (anom.) | F1-score (anom.) | PR-AUC (media prec.) | Rapporto su base casuale |
|---|---|---|---|---|---|
| Classificatore casuale (riferimento) | 0,000 | 0,000 | 0,000 | 0,049 (prevalenza 4,87%) | $1,0\times$ |
| **Isolation forest** | **0,124** | **0,127** | **0,126** | **0,105** | $\approx 2,2\times$ **sopra il caso** |
| **Autoencoder LSTM** | 0,096 | 0,098 | 0,097 | 0,067 | $\approx 1,4\times$ **sopra il caso** |

**Tabella 4.1:** Metriche di concordanza dei modelli non supervisionati rispetto alle regole cliniche sui 500 casi del livello gold.

I valori riportati nella Tabella 4.1 mostrano che **isolation forest** raggiunge una PR-AUC pari a **0,105** (circa $2,2$ volte superiore al riferimento casuale di $0,0487$, con F1-score pari a $0,126$), mentre l'**autoencoder LSTM** ottiene una PR-AUC di **0,067** (circa $1,4$ volte superiore al caso, con F1-score pari a $0,097$). L'entità contenuta di questi numeri in valore assoluto si spiega osservando la diversa natura dei segnali messi a confronto: le metriche non misurano la capacità dei modelli di replicare una diagnosi esterna, bensì il grado di sovrapposizione tra il $5\%$ di campioni giudicati più atipici nello spazio a otto dimensioni e gli istanti in cui scattano due disuguaglianze statiche su sole tre variabili. Mentre le regole cliniche si disattivano completamente non appena l'età della misura pressoria supera i $120$ secondi, i due modelli non supervisionati continuano ad analizzare la variabilità del battito (`HR_rolling_std`), le accelerazioni istantanee (`HR_delta`), la pressione diastolica e le sequenze di $30$ secondi, segnalando come anomale anche brusche aritmie o cadute pressorie che si arrestano appena sopra i $65$ mmHg. Il fatto che isolation forest risulti più allineato alle regole rispetto all'autoencoder deriva proprio dal fatto che entrambi valutano il singolo secondo puntuale, laddove la rete ricorrente giudica la ricostruzione dell'intera finestra di $30$ secondi. Se i modelli avessero mostrato una sovrapposizione prossima al $100\%$ con le regole, avrebbero rappresentato un semplice duplicato computazionalmente costoso di due condizioni elementari; la loro parziale divergenza conferma invece la complementarità dei quattro indicatori, illustrata dalla distribuzione degli errori di ricostruzione nella Figura 4.2 e dalla matrice di concordanza nella Figura 4.3.

Al termine dell'elaborazione, la funzione `save_anomalies_to_mongo(db, df_anomalies)` archivia nella collezione `anomalies_detected` esclusivamente i secondi in cui almeno uno dei quattro metodi ha segnalato un'anomalia, strutturando ciascun allarme nel documento BSON riportato nel Listato 4.2. Come mostrato, il documento indicizza l'intervento (`case_id`) e l'istante temporale (`timestamp`), registra nell'array `methods` l'elenco dei rilevatori che sono scattati simultaneamente (consentendo alla dashboard del Capitolo 5 di filtrare e colorare gli eventi per livello di gravità o per metodologia) e annota l'istante di calcolo `detected_at`. Per evitare che un'interruzione dello script lasci la collezione vuota o incoerente, invece di cancellare preventivamente i record pregressi viene adottato lo schema **insert-before-delete**: la funzione assegna a tutti i nuovi record

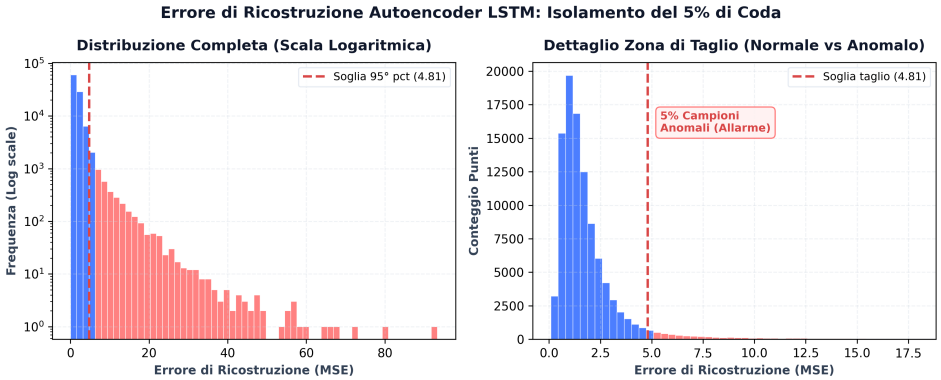



**Figura 4.2:** Distribuzione dell'errore di ricostruzione dell'autoencoder LSTM con soglia fissata al $95°$ percentile.

del lotto un unico valore `now = datetime.now(timezone.utc)`, li inserisce a blocchi di ventimila documenti tramite `insert_many` e, solo dopo il completamento di tutte le scritture, rimuove per i casi analizzati i documenti appartenenti a esecuzioni precedenti aventi `detected_at` diverso da `now`.

```
// Documento BSON archiviato nella collezione 'anomalies_detected'
{
  "_id": ObjectId("65d3f820e4b01a2b3c4d9a10"),
  "case_id": 1,
  "timestamp": ISODate("2020-01-01T08:24:15Z"),
  "methods": [
    "shock_index",
    "isolation_forest"
  ],
  "detected_at": ISODate("2026-10-17T17:08:51.711Z")
}
```

**Listato 4.2:** Struttura del documento BSON persistito nella collezione `anomalies_detected` con l'elenco dei rilevatori attivi.

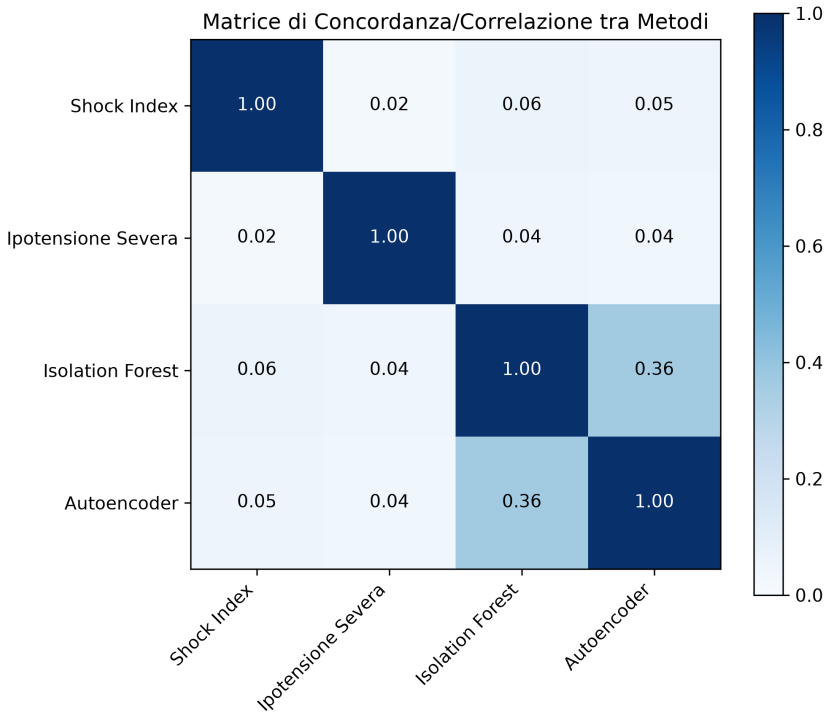



**Figura 4.3:** Matrice di concordanza tra le due regole cliniche deterministiche e i due modelli di machine learning.

Capitolo 5

# Ingegnerizzazione del sistema, interfaccia grafica e conclusioni

## 5.1 Servizi REST con FastAPI

Per mettere in comunicazione il livello gold di MongoDB, il motore ibrido di rilevamento delle anomalie e l'interfaccia utente, è stato realizzato un servizio web REST in Python basato su **FastAPI**, definito nel modulo `api/main.py` ed eseguito all'interno del container Docker `vitaldb_api` sulla porta `8000`. L'adozione di FastAPI ha permesso di dichiarare le rotte HTTP con validazione automatica dei tipi per i parametri di percorso e di query (come `case_id: int` e `window_seconds: Optional[int]`), di esporre la documentazione interattiva OpenAPI all'indirizzo `http://localhost:8000/docs` e di montare tramite `StaticFiles` nello stesso processo `uvicorn` anche le cartelle statiche `/dashboard` e `/presentazione`, evitando l'aggiunta di un terzo container dedicato a un server web esterno.

Sul piano della gestione delle connessioni, il modulo mantiene un riferimento globale `_client: Optional[MongoClient] = None` inizializzato al primo utilizzo dalla funzione `get_db()`, consentendo a tutte le richieste HTTP concorrenti di condividere il medesimo pool di connessioni TCP di PyMongo verso il container `vitaldb_mongo`. Inoltre, prima di serializzare le risposte per il client, il backend converte sistematicamente eventuali valori `np.nan` residui di Pandas in oggetti `None` (tradotti nello standard JSON come `null`), garantendo la piena compatibilità con il parser nativo dei browser web.

L'interfaccia REST si articola su sette endpoint principali: la rotta diagnostica `GET /health` verifica il collegamento con MongoDB tramite comando `ping` e restituisce i conteggi dei documenti presenti in `registry`, `vital_signals` e `anomalies_detected`; la rotta `GET /cases` restituisce la lista dei $500$ pazienti certificati con i relativi metadati anagrafici e di reparto; l'endpoint `GET /cases/{case_id}/series` fornisce la serie storica del singolo intervento a piena risoluzione ($1$ Hz) tramite `get_case_series` oppure ridotta su finestre temporali tramite `downsample_case` quando viene specificato il parametro `window_seconds`; le tre rotte `GET /quality`, `GET /benchmark` e `GET /evaluation` espongono infine i file JSON generati dagli script di analisi. A queste rotte di lettura si affianca l'endpoint `POST /cases/{case_id}/detect`, mostrato nel Listato 5.1, che avvia su richiesta l'analisi ibrida sul paziente selezionato invocando `run_detection_pipeline(db, [case_id])` e restituisce il riepilogo per metodo (`summary_by_method`) insieme al dettaglio cronologico dei secondi anomali.

```python
@app.post("/cases/{case_id}/detect")
def detect_case(case_id: int):
    """Esegue l'algoritmo multilivello di anomaly detection sul caso specificato."""
    db = get_db()
    df = run_detection_pipeline(db, [case_id])
    if df.empty:
        raise HTTPException(status_code=404, detail=f"Nessun dato per case_id={case_id}")

    flag_cols = [c for c in df.columns if c.endswith("_anomaly")]
    anomalous_rows = df[df[flag_cols].any(axis=1)]
    summary_by_method = {col.replace("_anomaly", ""): int(df[col].sum()) for col in flag_cols}

    def _val(row, col):
        return float(row[col]) if col in row and pd.notna(row[col]) else None

    anomalies = []
    for _, row in anomalous_rows.iterrows():
        methods = [c.replace("_anomaly", "") for c in flag_cols if bool(row[c])]
        si_val = _val(row, "shock_index")
        anomalies.append({
            "timestamp": str(row["timestamp"]),
            "methods": methods,
            "hr": _val(row, "Solar8000_HR"), "spo2": _val(row, "Solar8000_PLETH_SPO2"),
            "sbp": _val(row, "Solar8000_NIBP_SBP"), "dbp": _val(row, "Solar8000_NIBP_DBP"),
            "mbp": _val(row, "Solar8000_NIBP_MBP"),
            "shock_index": round(si_val, 3) if si_val is not None else None
        })

    return {
        "case_id": case_id,
        "anomaly_count": len(anomalies),
        "summary_by_method": summary_by_method,
        "anomalies": anomalies,
    }
```

**Listato 5.1:** Implementazione della rotta `POST /cases/{case_id}/detect` nel file `api/main.py`.

## 5.2 Dashboard web per l'esplorazione clinica e architetturale

L'interfaccia grafica del sistema, accessibile all'indirizzo `http://localhost:8000/dashboard`, è stata sviluppata utilizzando **HTML5, CSS3 e JavaScript standard** abbinati alla libreria **Chart.js** per la rappresentazione delle serie storiche, senza introdurre framework complessi come React o Angular che avrebbero richiesto una catena di compilazione Node.js dedicata dentro il container Docker. Per mantenere comunque il codice sorgente ordinato e diviso per responsabilità, lo script `dashboard/js/loader.js` carica dinamicamente all'avvio i frammenti HTML dichiarati tramite l'attributo `data-include` dentro `dashboard/index.html` (come la barra laterale, l'intestazione e le singole viste) utilizzando chiamate `fetch` parallele con parametro di versione `?v=3.9` per aggirare la cache del browser.

### 5.2.1 Vista di telemetria clinica e analisi del consenso

La prima vista dell'applicazione, gestita da `dashboard/js/app.js`, permette di ispezionare ciascuno dei $500$ interventi della coorte confrontando le segnalazioni delle regole cliniche e dei modelli di apprendimento automatico. Come mostrato nella Figura 5.1 relativa al caso intraoperatorio #62, la barra laterale sinistra elenca i pazienti con stato `committed` offrendo una casella di ricerca testuale e filtri rapidi per reparto chirurgico. La sezione centrale superiore espone i dati anagrafici del paziente selezionato (età $47$ anni, sesso femminile, chirurgia generale, $10.039$ punti nel livello gold), il selettore della risoluzione temporale (`Raw 1s`, `30s`, `60s` e `5m`), i valori medi riassuntivi dei cinque parametri vitali e soprattutto la matrice di consenso tra i quattro metodi di rilevamento, nella quale ogni scheda riporta il conteggio degli eventi individuati (sul caso #62: $330$ secondi per lo shock index, $0$ per l'ipotensione severa, $502$ per isolation forest e $501$ per l'autoencoder LSTM, per un totale di $1.060$ secondi anomali unici) e consente di filtrare visivamente i marker sui grafici sottostanti.

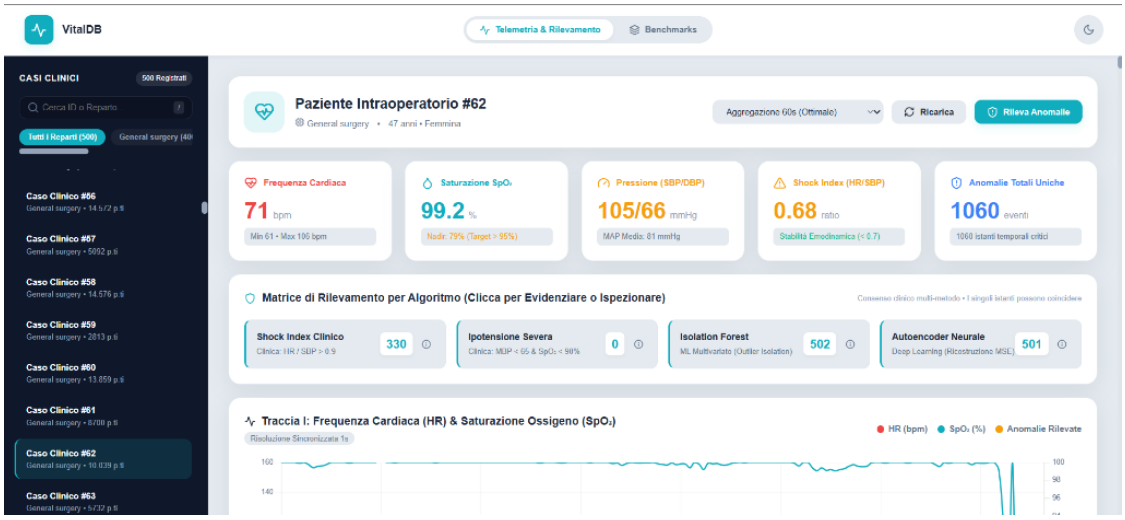
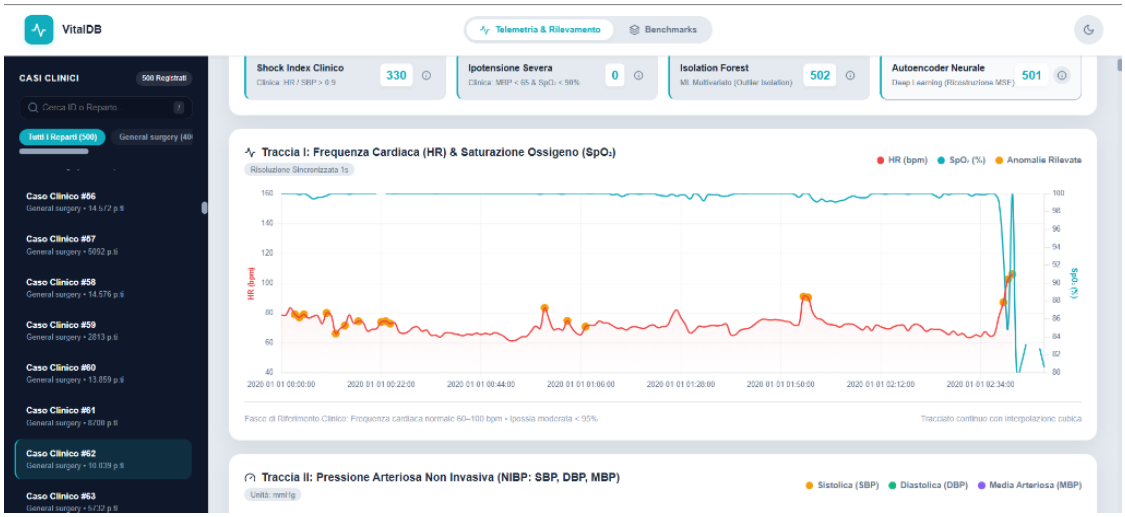



**Figura 5.1:** Quadro clinico del paziente e matrice di consenso a quattro metodi sulla dashboard web (caso #62).

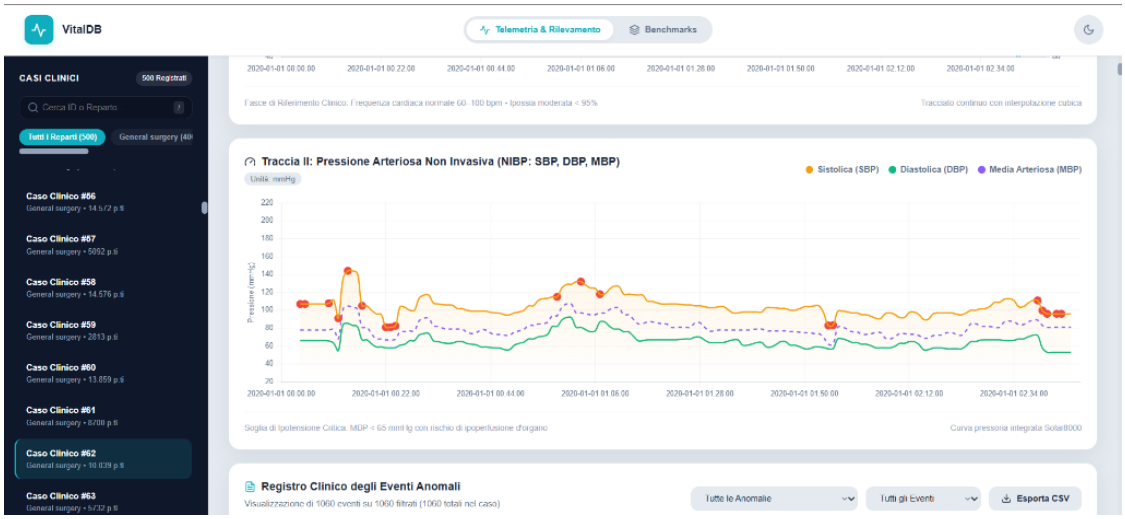
Subito sotto la matrice di consenso trovano posto i due grafici temporali sincronizzati illustrati nella Figura 5.2. Il primo grafico (Figura 5.2a) sovrappone l'andamento della frequenza cardiaca in rosso e della saturazione d'ossigeno in azzurro evidenziando gli istanti anomali con indicatori circolari dorati; il secondo grafico (Figura 5.2b) riporta l'evoluzione delle tre componenti pressorie (`SBP`, `DBP` e `MBP`) rispetto alla linea di riferimento clinico di $65$ mmHg. La vista si chiude nella parte inferiore con il registro tabellare degli eventi anomali mostrato nella Figura 5.3, che elenca in ordine cronologico tutti gli istanti segnalati classificandoli per livello di severità in base al numero di algoritmi concordi (**advisory** con un solo metodo attivo, **warning** con due metodi concordi e **critical** con tre o quattro metodi) e permette di scaricare l'intero prospetto in formato CSV.

### 5.2.2 Vista di monitoraggio dell'architettura e dei benchmark

La seconda vista della dashboard, gestita dallo script `dashboard/js/lakehouse.js`, interroga gli endpoint `/benchmark`, `/evaluation` e `/quality` per esporre le metriche ingegneristiche dell'ultimo ciclo di esecuzione della pipeline. I pannelli superiori presentano il confronto sullo spazio occupato dai tre livelli (**33,95 MB** in bronze, **44,52 MB** in silver e **15,60 MB** di dati compressi in gold oltre a **2,56 MB** di indici B-Tree), dimensionando le barre grafiche rispetto al valore massimo tramite `Math.max(bronzeMB, silverMB, goldMB)`, insieme al confronto sui tempi di risposta (**12,02 ms** su MongoDB contro **26,95 ms** su Parquet). I pannelli inferiori espongono la tabella con le metriche di valutazione dei modelli non supervisionati sui $5.402.386$ campioni e il prospetto sulla




**(a)** Andamento di frequenza cardiaca e saturazione d'ossigeno con evidenziazione degli istanti anomali.




**(b)** Andamento della pressione arteriosa sistolica, diastolica e media rispetto alla soglia di rischio.

**Figura 5.2:** Grafici temporali sincronizzati sulla dashboard web per l'ispezione del tracciato a risoluzione di un secondo.

qualità della bonifica, che rendiconta le $5.565.384$ righe grezze iniziali, le $162.998$ righe inattive rimosse $(2,93\%)$ e il dettaglio di outlier e valori mancanti per ciascun intervento.

## 5.3 Containerizzazione Docker, orchestrazione e test automatici

Per assicurare che l'intero ambiente sia riproducibile su qualsiasi sistema operativo senza richiedere l'installazione manuale di MongoDB 8.0 o di librerie scientifiche locali, la piattaforma è stata incapsulata in due container gestiti da `docker-compose.yml`.

Sulla rete virtuale isolata `vitaldb_net` operano il container `vitaldb_mongo`, basato sull'immagine ufficiale `mongo:8.0` avviata con l'opzione `-replSet rs0` e dotata di una sonda di controllo (`healthcheck`) che verifica o inizializza automaticamente il replica set tramite `mongosh`, e il container `vitaldb_api`, costruito a partire da `python:3.12-slim` con tutte le dipendenze di `requirements.txt` (inclusa la distribuzione CPU di PyTorch). Grazie alla direttiva `depends_on` vincolata alla condizione `service_healthy`, il servizio FastAPI attende che il motore MongoDB abbia completato l'elezione del nodo primario prima di accettare richieste sulla porta `8000`.

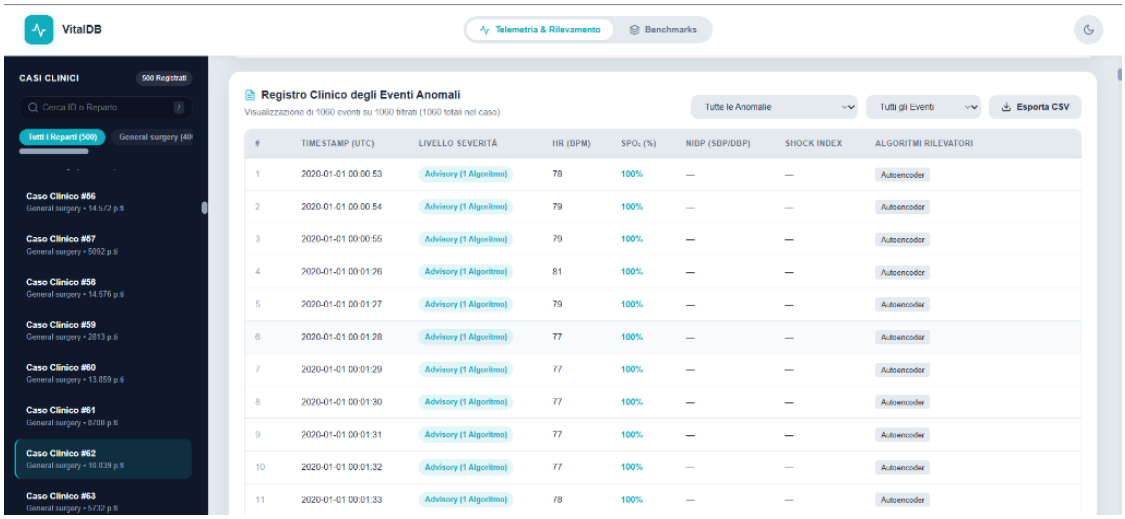



**Figura 5.3:** Registro cronologico degli eventi anomali con indicazione della severità, dei parametri vitali e funzione di esportazione CSV.

L'esecuzione ordinata dei sei stadi della pipeline sui $500$ casi clinici — scaricamento bronze (`download.py`), pulizia silver (`clean.py`), caricamento gold (`load_mongo.py`), rilevamento delle anomalie (`detector.py`), benchmark delle prestazioni (`benchmark_etl.py`) e valutazione dei modelli (`evaluate_models.py`) — è governata dallo script orchestratore `scripts/run_pipeline.py`, avviabile con un unico comando:

```
docker exec vitaldb_api python -u scripts/run_pipeline.py
```

L'orchestratore lancia i sei moduli in sequenza controllando il codice di uscita di ciascun sottoprocesso e interrompe immediatamente la catena qualora una fase restituisca un errore (`exit code != 0`), evitando disallineamenti tra i file su disco e le collezioni del database. A supporto dei cicli di sviluppo è stato inoltre realizzato lo script `scripts/reset_pipeline.py`, che previa conferma (`--yes`) svuota le collezioni MongoDB e rimuove i file Parquet e i report JSON per consentire una ripartenza da zero.

La correttezza funzionale dell'intero codice è presidiata da una suite di **38 test automatici** eseguibili tramite `pytest` e distribuiti in sei moduli nella cartella `tests/`, riepilogati nella Tabella 5.1. In particolare, nei test dell'endpoint `POST /cases/{id}/detect` dentro `tests/test_api.py` viene sostituito dinamicamente il metodo di scrittura `save_anomalies_to_mongo` tramite `monkeypatch`, così da verificare l'intera catena di calcolo senza sovrascrivere nel database le anomalie reali calcolate sulla coorte di $500$ pazienti.

## 5.4 Conclusioni, analisi dei limiti e sviluppi futuri

Il lavoro svolto ha permesso di progettare, implementare e validare su una coorte reale di **500 interventi chirurgici** estratti da VitalDB (**5.565.384 secondi grezzi** nel livello bronze e **5.402.386 secondi bonificati** nei livelli silver e gold) una piattaforma completa di ingegneria dei dati e rilevamento automatico delle anomalie. I risultati sperimentali confermano anzitutto il valore pratico della separazione dei ruoli nell'architettura medallion: conservare i tracciati originali in Apache Parquet nel livello bronze (**33,95 MB**) ha svincolato lo sviluppo dai tempi di rete dell'API remota; utilizzare il livello silver (**44,52 MB**, $+31,1\%$) per arricchire le tabelle con dodici colonne di qualità e ridurre i valori mancanti dal $66,3\%$ al $26,3\%$ senza cancellare gli outlier ha garantito una tracciabilità trasparente; infine, modellare il livello gold sulle collezioni temporali di MongoDB 8.0 ha compresso i dati fisici a **15,60 MB** ($-65,0\%$ rispetto a silver, oltre a **2,56 MB** di indici B-Tree)

| Modulo di test | Numero test | Ambito di verifica e garanzie controllate |
|---|---|---|
| `tests/test_config.py` | 2 | Coerenza degli intervalli fisiologici in `TRACK_SPECS` ($\min < \max$) e verifica che la soglia clinica `NIBP_MAX_CLINICAL_AGE_S` ($120$ s) non superi il limite LOCF ($300$ s). |
| `tests/test_bronze.py` | 3 | Scrittura atomica tramite file temporaneo (`.tmp` $\rightarrow$ `.parquet`), integrità delle colonne scaricate e generazione di `download_manifest.json` con `run_id`. |
| `tests/test_etl.py` | 4 | Interpolazione lineare limitata ai soli buchi interni $\leq 5$ s, calcolo degli outlier prima dell'imputazione, tracciamento dei cinque flag `*_imputed` e verifica dell'identità `observed` $+$ `imputed` $+$ `missing` $=$ `rows_cleaned`. |
| `tests/test_gold.py` | 16 | Conversione in documenti BSON con omissione dei campi `NaN`, validazione `validate_records`, struttura del `registry`, filtro sui soli casi `committed` e recupero automatico `recover_incomplete_loads`. |
| `tests/test_detection.py` | 9 | Calcolo di shock index e ipotensione severa, blocco degli allarmi con `nibp_age_s` $> 120$ s, isolamento delle finestre scorrevoli tra pazienti diversi, presenza del proiettore `nn.Linear` e assenza di contaminazione tra casi nel `.bfill()` raggruppato. |
| `tests/test_api.py` | 4 | Collaudo tramite `TestClient` delle rotte `/health`, `/cases`, `/cases/{id}/series` e `POST /cases/{id}/detect` con isolamento del database tramite `monkeypatch`. |

**Tabella 5.1:** Riepilogo dei 38 test unitari e di integrazione sviluppati con `pytest` nella cartella `tests/`.

dimezzando i tempi di risposta sulle interrogazioni per singolo paziente (**12,02 ms** contro **26,95 ms**, pari a un fattore **2,2**$\times$). Alrettanto determinante si è rivelata la cura nella preparazione del dato per gli algoritmi a valle: rilevare gli outlier prima della propagazione in avanti, distinguere l'orizzonte di mantenimento pressorio nel livello silver ($300$ s) dal vincolo di freschezza per le regole cliniche ($120$ s) e confinare il `.bfill()` dentro ciascun `case_id` hanno evitato distorsioni sistematiche nei modelli.

Accanto ai risultati raggiunti, un'analisi critica dell'architettura realizzata consente di mettere in luce con trasparenza i limiti progettuali e operativi del sistema. Il limite strutturale primario risiede nella natura esclusivamente **batch e retrospettiva** dell'intera pipeline: l'infrastruttura presuppone che l'intervento chirurgico sia già interamente concluso e archiviato su file prima di avviare le fasi di bonifica, caricamento e rilevamento. In un contesto clinico reale di sala operatoria o di terapia intensiva, tuttavia, i parametri vitali vengono generati come flussi continui secondo per secondo ($1$ Hz): l'attuale implementazione non dispone di un layer di ingestione in streaming (ad esempio tramite broker come Apache Kafka o protocolli IoT come MQTT) né di un motore di elaborazione su code temporali scorrevoli dal vivo (come Apache Flink), precludendo l'emissione di allarmi istantanei al secondo esatto in cui l'evento avverso si manifesta al monitor del paziente.

Per quanto riguarda i livelli **bronze e silver**, il controllo sui file già presenti su disco prima di applicare lo stato `skipped_existing` verifica la presenza delle colonne e il passo di campionamento tra i primi due secondi senza ricalcolare un'impronta crittografica dell'intero file Parquet, mentre i referti salvati in `lab_data.parquet` vengono storicizzati ma non sono ancora integrati con le serie a $1$ Hz. Nel livello silver, inoltre, la scelta di non rimuovere fisicamente i campioni marcati come

`*_outlier` fa sì che la loro esclusione effettiva avvenga solo al momento della lettura in `load_from_gold`, e non è ancora previsto un filtro bloccante a livello di intero paziente (**quality gate**) che escluda dal caricamento nel livello gold gli interventi caratterizzati da una copertura sensoriale molto bassa.

Sul fronte del **livello gold e del data lakehouse applicativo**, le restrizioni native delle collezioni temporali di MongoDB 8.0 — che non supportano transazioni multi-documento verso il `registry` né validatori `$jsonSchema` su `vital_signals` — implicano che l'atomicità del caricamento a due fasi e il controllo dei tipi sulle misure dipendano dal passaggio attraverso gli script Python e dalla procedura di recupero `recover_incomplete_loads`, anziché essere imposti dal motore del database su qualsiasi scrittura esterna. Allo stesso modo, l'identificativo di esecuzione `run_id` è registrato nel catalogo `registry` e nei quattro file JSON senza essere replicato dentro ciascuno dei $5,4$ milioni di documenti temporali, l'asse del tempo parte dalla data convenzionale del $1°$ gennaio $2020$ poiché VitalDB fornisce solo secondi relativi anonimizzati, e l'interrogazione `GET /cases` richiede ancora uno stadio di raggruppamento `$group` su `vital_signals` per recuperare reparto, età e sesso dei pazienti. Anche i risultati del benchmark di latenza vanno letti nel loro contesto specifico: il vantaggio di $2,2\times$ misura l'interrogazione selettiva di un singolo paziente con dati caldi in memoria su macchina singola, mentre su scansioni analitiche dell'intera coorte senza filtri la lettura colonnare diretta dei file Parquet rimane altamente competitiva.

Relativamente al modulo di **anomaly detection e ai servizi applicativi**, l'impostazione di `contamination=0.05` in isolation forest e del $95°$ percentile nell'autoencoder LSTM fissa per costruzione al $5\%$ la frazione di campioni segnalati, configurando i due algoritmi come meccanismi di ordinamento per severità anziché come stimatori autonomi della prevalenza patologica, mentre la valutazione tramite PR-AUC ($0,105$ e $0,067$) misura la concordanza interna rispetto alle regole cliniche in assenza di diagnosi mediche indipendenti. Inoltre, il sistema attuale non salva su disco i pesi dei modelli addestrati sull'intera coorte (`.pkl` e `.pt`) e non espone una rotta `GET /cases/{id}/anomalies` di sola lettura: quando l'utente avvia il rilevamento dalla dashboard tramite `POST /cases/{id}/detect`, la pipeline riaddestra i modelli sul solo paziente selezionato e aggiorna per quel caso i record in `anomalies_detected`. Infine, sul piano dell'interfaccia web, la vista architetturale limita la tabella di dettaglio ai primi $15$ pazienti per mantenere reattivo il browser e non espone ancora nell'intestazione grafica gli identificativi `run_id` e `pipeline_version`.

Questi stessi limiti tracciano in modo diretto le principali direttrici di sviluppo futuro del progetto: la serializzazione su disco dello `StandardScaler`, del modello `IsolationForest` e dei pesi PyTorch dell'autoencoder addestrati sui $500$ casi, separando nell'API la consultazione immediata degli allarmi precalcolati (`GET /cases/{id}/anomalies`) dal ricalcolo esplicito; l'inserimento nel documento del catalogo `registry` dei campi anagrafici (`department`, `age`, `sex`) per servire la rotta `GET /cases` senza interrogare `vital_signals`, affiancato da un identificativo `detection_run_id` nei record di anomalia; l'introduzione nel livello silver di una soglia minima di copertura per paziente prima della promozione a gold; e infine l'evoluzione verso un'architettura ibrida (*streaming* e *batch*) mediante code a eventi (come Apache Kafka o MQTT) per consentire il monitoraggio dal vivo durante l'intervento chirurgico, affiancata dal passaggio dello script `run_pipeline.py` a un orchestratore come Apache Airflow o Prefect ed estendendo l'analisi anche alla pressione arteriosa invasiva (`ART`), alla capnografia ($EtCO_2$) e all'indice bispettrale (`BIS`).